

فصل ۱۰: حافظه مجازی (Virtual Memory)

۱۰.۲ ساختار و ساختارهای کنترلی (Hardware and Control Structures)

توضیح ساده و قابل فهم

این اسلاید درباره نحوه مدیریت حافظه مجازی در سیستم عامل است و بر سه مفهوم کلیدی تمرکز دارد:

۱. آدرس‌های منطقی و فیزیکی

- هر دسترسی به حافظه توسط یک فرآیند، با استفاده از آدرس منطقی (Logical Address) انجام می‌شود.
- آدرس منطقی به طور دینامیک و در زمان اجرا به آدرس فیزیکی (Physical Address) ترجمه می‌شود.
- این ترجمه به سیستم عامل اجازه می‌دهد که فرآیندها را در مکان‌های متفاوت حافظه اصلی جا به جا کند بدون آنکه فرآیند متوجه شود.

۲. تقسیم فرآیند به صفحات یا بخش‌ها

- یک فرآیند می‌تواند به چندین تکه کوچک (صفحه یا بخش) تقسیم شود.
- این تکه‌ها لازم نیست که به صورت متوالی در حافظه اصلی قرار گیرند.
- با استفاده از جدول صفحات (Page Table) یا جدول بخش‌ها (Segment Table)، سیستم عامل می‌تواند آدرس منطقی را به آدرس فیزیکی مناسب تبدیل کند.

۳. لزوم قرار داشتن همه صفحات یا بخش‌ها در حافظه اصلی

- با وجود ویژگی‌های بالا، نیازی نیست که همه صفحات یا بخش‌های یک فرآیند به طور همزمان در حافظه اصلی باشند.
- این امکان باعث می‌شود حافظه اصلی به صورت بهینه استفاده شود و فرآیندها بتوانند حتی با بخشی از حافظه اصلی اجرا شوند (که اساس مفهوم حافظه مجازی demand paging است).

مثال آموزشی ساده

فرض کنید یک فرآیند داریم که شامل ۴ صفحه است و حافظه اصلی فقط جا برای ۲ صفحه دارد:

صفحه فرآیند	آدرس منطقی	وضعیت در حافظه اصلی
صفحه ۰	۰-۴ کیلوبایت	در حافظه اصلی
صفحه ۱	۴-۸ کیلوبایت	در حافظه اصلی
صفحه ۲	۸-۱۲ کیلوبایت	روی دیسک
صفحه ۳	۱۲-۱۶ کیلوبایت	روی دیسک

- وقتی CPU نیاز دارد به صفحه ۲ دسترسی پیدا کند، سیستم عامل آن را از دیسک به حافظه اصلی منتقل می‌کند و ممکن است یکی از صفحات فعلی را خارج کند (Page Replacement).
- فرآیند بدون اینکه از این جابه‌جایی مطلع شود، ادامه می‌دهد.

نکات کلیدی

- آدرس منطقی $\neq$ آدرس فیزیکی؛ همه دسترسی‌ها با آدرس منطقی انجام می‌شود و در زمان اجرا ترجمه می‌شوند.
- فرآیندها می‌توانند در مناطق غیرمتوالی حافظه اصلی قرار گیرند.
- حافظه اصلی لازم نیست کل فرآیند را نگه دارد؛ سیستم عامل می‌تواند صفحات/بخش‌ها را به صورت دینامیک بارگذاری کند.
- این ویژگی پایه‌ی حافظه مجازی (Virtual Memory) است که امکان اجرای فرآیندها با حافظه کمتر از اندازه واقعی‌شان را می‌دهد.

اجرای برنامه (Run Program) – حافظه مجازی

توضیح ساده و قابل فهم

این اسلاید به نحوه اجرای یک فرآیند در حافظه مجازی و مفهوم Resident Set می‌پردازد:

1. Resident Set (مجموعه مقیم)

- به بخش‌هایی از یک فرآیند گفته می‌شود که در هر لحظه در حافظه اصلی قرار دارند Resident Set.
- این مجموعه ممکن است فقط یک قسمت از کل فرآیند باشد، نه همه آن.

2. Page Fault (آدرس حافظه)

- وقتی پردازنده به آدرس منطقی‌ای دسترسی پیدا می‌کند که در حافظه اصلی نیست، یک Interrupt تولید می‌شود که به آن Memory Access Fault یا Page Fault گفته می‌شود.

- سیستم عامل این فرآیند را در حالت مسدود (Blocking) قرار می‌دهد و کنترل CPU را به دست می‌گیرد.

3. بارگذاری صفحه و ادامه اجرا

- سیستم عامل صفحه یا بخش مورد نیاز را از دیسک به حافظه اصلی منتقل می‌کند (I/O Operation).
- وقتی صفحه بارگذاری شد، یک I/O Interrupt تولید می‌شود و سیستم عامل فرآیند را به حالت Ready باز می‌گرداند تا CPU بتواند اجرای آن را ادامه دهد.

4. مزایای حافظه مجازی

- امکان نگه داشتن تعداد بیشتری فرآیند در حافظه اصلی.
- امکان اجرای فرآیندهایی که بزرگ‌تر از حافظه اصلی هستند.

مثال آموزشی ساده

فرض کنید فرآیند P شامل ۴ صفحه است و فقط صفحات ۰ و ۱ در حافظه اصلی قرار دارند (Resident Set). CPU نیاز دارد به صفحه ۲ دسترسی پیدا کند:

- رخ می‌دهد Page Fault → به صفحه ۲ دسترسی می‌خواهد CPU.
- سیستم عامل فرآیند را مسدود می‌کند.
- صفحه ۲ از دیسک به حافظه اصلی منتقل می‌شود.
- سیستم عامل فرآیند را به حالت Ready باز می‌گرداند.
- اجرای فرآیند را ادامه می‌دهد CPU.

نمودار Mermaid: جریان Page Fault و بازگشت به Ready

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- بخشی از فرآیند که در هر لحظه در حافظه اصلی قرار دارد: Resident Set.
- به صفحه‌ای که در حافظه نیست دسترسی می‌خواهد CPU زمانی رخ می‌دهد که: Page Fault.
- سیستم عامل فرآیند را موقتاً مسدود می‌کند و بعد از بارگذاری صفحه آن را دوباره آماده می‌کند: Blocking و Ready.
- مزایای حافظه مجازی:
 - نگهداری تعداد بیشتری فرآیند در حافظه اصلی.
 - امکان اجرای فرآیندهایی که بزرگ‌تر از حافظه اصلی هستند.

در حافظه مجازی Thrashing

توضیح ساده و قابل فهم

این اسلاید درباره پدیده‌ای به نام Thrashing در حافظه مجازی صحبت می‌کند:

1. تعریف Thrashing

- وقتی سیستم عامل یک صفحه یا بخش از فرآیند را بارگذاری می‌کند، ممکن است مجبور شود یک صفحه دیگر را از حافظه اصلی خارج کند تا جای صفحه جدید باز شود.
- اگر سیستم صفحه‌ای را خارج کند که قرار است خیلی زود دوباره استفاده شود، آن صفحه باید دوباره بارگذاری شود.
- این چرخه دائمی بارگذاری و خارج کردن صفحات، بدون اینکه فرآیندها دستورالعمل‌ها را اجرا کنند، Thrashing نامیده می‌شود.

2. نشانه‌های Thrashing

- زمان CPU برای اجرای دستورالعمل‌ها به شدت کاهش می‌یابد.
- سیستم اکثر وقت خود را صرف Swap کردن صفحات بین حافظه اصلی و دیسک می‌کند.
- در نتیجه، کارایی سیستم بسیار پایین می‌آید.

مثال آموزشی ساده

فرض کنید حافظه اصلی جا برای ۲ صفحه دارد و یک فرآیند نیاز به ۳ صفحه دارد. روند اجرا به صورت زیر است:

گام	صفحات در حافظه	عملیاتی که رخ می‌دهد
۱	صفحه ۰، ۱	به صفحه ۲ نیاز دارد → صفحه ۰ خارج می‌شود، صفحه ۲ بارگذاری می‌شود CPU
۲	صفحه ۰، ۲	به صفحه ۱ نیاز دارد → صفحه ۱ خارج می‌شود، صفحه ۰ بارگذاری می‌شود CPU
۳	صفحه ۰، ۲	به صفحه ۱ نیاز دارد → صفحه ۲ خارج می‌شود، صفحه ۱ بارگذاری می‌شود CPU

- مشاهده می‌کنیم که سیستم فقط صفحات را جابه‌جا می‌کند و اجرای واقعی دستورالعمل‌ها بسیار کم است.
- این همان Thrashing است.

نمودار Mermaid: چرخه Thrashing

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

- این نمودار نشان می‌دهد که در Thrashing سیستم در چرخه مکرر بارگذاری و خارج کردن صفحات گیر می‌کند.

نکات کلیدی

- زمانی رخ می‌دهد که سیستم بیش از حد صفحات را جابه‌جا می‌کند: Thrashing.
- سیستم در حالت Thrashing بیشتر زمانش را صرف مدیریت حافظه می‌کند تا اجرای واقعی دستورالعمل‌ها.
- علت اصلی: اندازه Resident Set یک فرآیند کمتر از حد نیاز واقعی آن است.
- راه حل‌ها: افزایش اندازه حافظه، کاهش تعداد فرآیندهای همزمان یا استفاده از الگوریتم‌های جایگزینی صفحات بهتر.

اصل محلیت (Principle of Locality) در حافظه مجازی

توضیح ساده و قابل فهم

این اسلاید به یکی از اصول پایه حافظه مجازی به نام **Principle of Locality** می‌پردازد که به سیستم عامل کمک می‌کند **thrashing** را کاهش دهد:

1. تعریف Principle of Locality

- در طول اجرای یک فرآیند، دسترسی‌ها به دستورالعمل‌ها و داده‌ها تمایل دارند در نواحی مشخصی متمرکز شوند.
- به عبارت دیگر، برنامه‌ها و داده‌ها معمولاً به صورت خوشه‌ای (**clustered**) مورد دسترسی قرار می‌گیرند.

2. انواع مختلف Locality دارد

- اگر یک داده یا دستورالعمل در حال حاضر استفاده شده، احتمال استفاده مجدد آن در کوتاه‌مدت (**Temporal Locality** (محلیت زمانی): زیاد است.
- اگر یک داده یا دستورالعمل استفاده شود، احتمال استفاده از داده‌ها یا دستورالعمل‌های مجاور آن (**Spatial Locality** (محلیت مکانی): نیز زیاد است.

3. اهمیت Principle of Locality

- این اصل به سیستم عامل اجازه می‌دهد حدس‌های هوشمندانه‌ای درباره صفحات مورد نیاز در آینده نزدیک بزند.
- نتیجه: کاهش **Page Fault** و جلوگیری از **Thrashing**.

مثال آموزشی ساده

فرض کنید فرآیند زیر در حال اجرا است:

Instruction sequence: I1, I2, I3, I4, I5, I6
Data sequence: D1, D2, D3

- اگر صفحه‌ای که شامل I1 و I2 است در حافظه باشد، احتمالاً I3 و I4 هم به زودی نیاز خواهند شد → سیستم عامل می‌تواند پیش‌بینی کند و صفحه بعدی را آماده کند.
- همینطور برای داده‌ها: اگر D1 استفاده شد، احتمال استفاده D2 و D3 نیز زیاد است.

این پیش‌بینی باعث می‌شود صفحات درست بارگذاری شوند و **Thrashing** رخ ندهد.

نمودار Mermaid: اصل محلیت در حافظه

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

- نمودار نشان می‌دهد که سیستم عامل با استفاده از محلیت زمانی و مکانی، می‌تواند صفحات مورد نیاز را پیش‌بینی کند.

نکات کلیدی

- Principle of Locality:** دسترسی‌های برنامه و داده معمولاً خوشه‌ای هستند.
- Temporal Locality:** استفاده دوباره از یک صفحه در کوتاه‌مدت محتمل است.
- Spatial Locality:** استفاده از صفحات یا داده‌های مجاور محتمل است.

- استفاده از این اصل باعث کاهش Page Fault و جلوگیری از Thrashing می‌شود.
- سیستم عامل می‌تواند با بارگذاری هوشمند صفحات، کارایی حافظه مجازی را افزایش دهد.

Paging در حافظه مجازی (صفحه‌بندی)

توضیح ساده و قابل فهم

این اسلاید به مکانیزم Paging در حافظه مجازی می‌پردازد:

- جدول صفحات (Page Table)
 - هر فرآیند جدول صفحات مخصوص خودش را دارد.
 - وقتی همه صفحات یک فرآیند در حافظه اصلی قرار می‌گیرند، جدول صفحات آن فرآیند نیز ایجاد و در حافظه اصلی بارگذاری می‌شود.
 - کاربرد جدول صفحات: ترجمه آدرس‌های منطقی به آدرس‌های فیزیکی.
- شماره قاب (Frame Number)
 - هر خانه در جدول صفحات (Page Table Entry) شامل شماره قاب (Frame Number) است که نشان می‌دهد صفحه مربوطه در کدام بخش از حافظه اصلی قرار دارد.
- بیت حضور (Present Bit – P)
 - چون ممکن است فقط بخشی از صفحات یک فرآیند در حافظه اصلی باشند، یک بیت در جدول صفحات لازم است تا مشخص شود که آیا صفحه در حافظه اصلی است یا خیر.
- بیت تغییر (Modify Bit – M)
 - این بیت مشخص می‌کند که آیا صفحه از آخرین بار که به حافظه اصلی بارگذاری شده، تغییر کرده است یا نه.
 - اگر تغییر کرده باشد و قرار باشد از حافظه خارج شود، باید ابتدا به دیسک بازگردد (Write Back).

مثال آموزشی ساده

فرض کنید یک فرآیند با ۴ صفحه داریم و جدول صفحات آن به صورت زیر است:

Page Number	Frame Number	P (Present)	M (Modified)
0	5	1	0
1	9	1	1
2	–	0	0
3	–	0	0

- صفحه 0 و 1 در حافظه اصلی هستند ($P=1$) و صفحه 1 تغییر کرده است ($M=1$).
- صفحه 2 و 3 در حافظه اصلی نیستند ($P=0$). اگر CPU به این صفحات نیاز داشته باشد، Page Fault رخ می‌دهد.

نمودار Mermaid: ساختار جدول صفحات

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

- هر فرآیند یک جدول صفحات مخصوص دارد.
- جدول صفحات شامل شماره قاب، بیت حضور و بیت تغییر برای هر صفحه است.

نکات کلیدی

- هر فرآیند جدول صفحات مخصوص خود را دارد.
- جدول صفحات برای ترجمه آدرس منطقی به فیزیکی استفاده می‌شود.
- **Present Bit (P)**: مشخص می‌کند که صفحه در حافظه اصلی است یا خیر.
- **Modify Bit (M)**: مشخص می‌کند که صفحه تغییر کرده و نیاز به نوشتن روی دیسک دارد یا نه.
- این ساختار پایه‌ای است برای **Paging و مدیریت حافظه مجازی** و جلوگیری از بارگذاری غیرضروری صفحات.

Virtual address

Page number	Offset
-------------	--------

Page table entry

P	M	Other control bits	Frame number
---	---	--------------------	--------------

(a) Paging only

Virtual address

Segment number	Offset
----------------	--------

Segment table entry

P	M	Other control bits	Length	Segment base
---	---	--------------------	--------	--------------

(b) Segmentation only

Virtual address

Segment number	Page number	Offset
----------------	-------------	--------

Segment table entry

Control bits	Length	Segment base
--------------	--------	--------------

Page table entry

P	M	Other control bits	Frame number
---	---	--------------------	--------------

P = present bit
M = modified bit

(c) Combined segmentation and paging

ساختار جدول صفحه (Page Table Structure)

این اسلاید سه مدل اصلی نگاشت آدرس مجازی به آدرس فیزیکی را نشان می‌دهد:

۱. صفحه‌بندی (Paging)
۲. بخش‌بندی (Segmentation)
۳. ترکیب بخش‌بندی و صفحه‌بندی (Segmentation + Paging)

1. Paging Only

- آدرس مجازی شامل: **Page Number + Offset**
- هر ورودی جدول صفحه (Page Table Entry):

- **P (Present bit):** آیا صفحه در حافظه هست؟
- **M (Modified bit):** آیا صفحه تغییر کرده؟
- **Frame number:** محل قرارگیری صفحه در (شماره قاب RAM).

نمودار:

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

2. Segmentation Only

- آدرس مجازی شامل: **Segment Number + Offset**
- هر ورودی جدول بخش (Segment Table Entry):

- **P, M:** بیت‌های کنترلی
- **Length:** طول بخش
- **Segment Base:** آدرس شروع بخش

نمودار:

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

3. Combined Segmentation + Paging

- آدرس مجازی شامل: **Segment Number + Page Number + Offset**
- مراحل:

1. **Segment Table** مشخص می‌کند هر بخش از کجا شروع می‌شود.
2. در داخل هر بخش، **Page Table** تعیین می‌کند صفحه در کدام قاب RAM است.
- ترکیب انعطاف‌پذیری segmentation با سادگی paging.

نمودار:

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

مثال ساده

فرض کنید برنامه‌ای سه بخش دارد: کد، داده، استک.

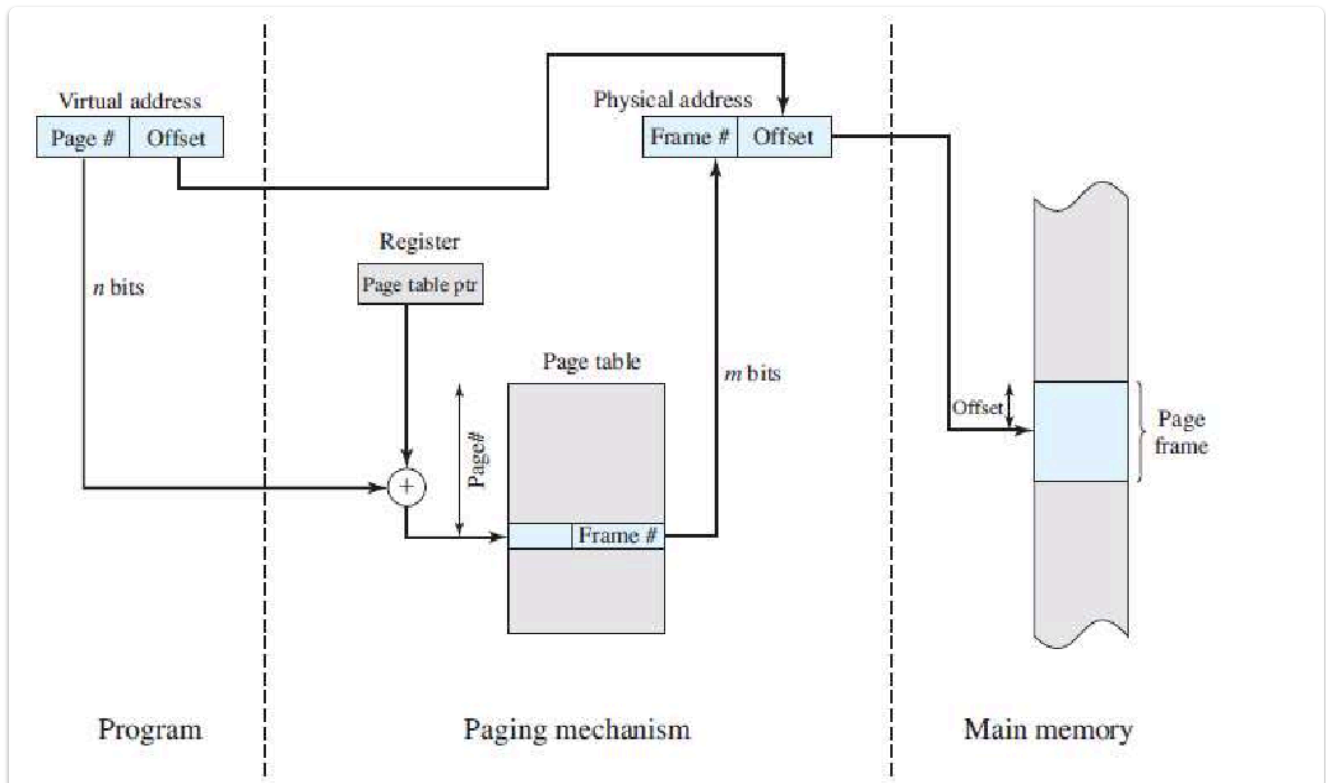
- با **Segmentation** هر بخش جدا نگهداری می‌شود.
- با **Paging** هر بخش به صفحات کوچک تقسیم شده و پراکنده در حافظه ذخیره می‌شود.
- با **Segmentation + Paging**: ابتدا بخش انتخاب می‌شود، سپس صفحه داخل آن بخش.

نکات کلیدی

- **Paging**: تقسیم حافظه به بلوک‌های یکسان → ساده ولی بدون توجه به منطق برنامه.
- **Segmentation**: تقسیم بر اساس ساختار منطقی برنامه (کد، داده، استک).
- **Segmentation + Paging**: ترکیب هر دو → مدیریت بهینه‌تر حافظه.

• بیت‌های مهم:

- **P (Present)**: صفحه/بخش در حافظه است یا نه.
- **M (Modified)**: تغییر یافته و نیاز به ذخیره روی دیسک دارد.



Address Translation (ترجمه آدرس)

این اسلاید مکانیزم تبدیل **آدرس مجازی** (Virtual Address) به **آدرس فیزیکی** (Physical Address) در سیستم‌های صفحه‌بندی را نشان می‌دهد.

مراحل ترجمه

1. **برنامه** یک آدرس مجازی تولید می‌کند:
• شامل دو بخش:

- **Page Number** (شماره صفحه)
- **Offset** (جابه‌جایی داخل صفحه)

2. **رجیستر پایه جدول صفحه** (Page Table Base Register) → مکان شروع جدول صفحه را نگه می‌دارد.

3. **Page Number** + مقدار رجیستر پایه → آدرس ورودی مربوط در جدول صفحه

4. جدول صفحه مقدار **Frame Number** (شماره قاب در حافظه اصلی) را برمی‌گرداند.

5. با ترکیب **Frame Number + Offset** → آدرس فیزیکی ساخته می‌شود.

6. در نهایت این آدرس در **Main Memory** به داده واقعی اشاره می‌کند.

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

مثال ساده

- فرض کنید آدرس مجازی = (Page=2, Offset=50)
- جدول صفحه نشان می‌دهد که **Page 2 → Frame 7**
- آدرس فیزیکی = (Frame 7, Offset 50)

نکات کلیدی

- Virtual Address = Page # + Offset**
- Physical Address = Frame # + Offset**
- جدول صفحه (Page Table) نقش نگاشت را دارد.
- رجیستر پایه جدول صفحه مشخص می‌کند جدول در حافظه کجاست.

Process Page Table (جدول صفحه هر فرآیند)

توضیح

- در اغلب سیستم‌ها، هر فرآیند یک جدول صفحه (Page Table) جداگانه دارد.
- چون هر فرآیند می‌تواند فضای مجازی بسیار بزرگی داشته باشد، جدول صفحه ممکن است بسیار بزرگ شود.

مثال: معماری VAX

- فضای آدرس مجازی هر فرآیند: $2^{31} = 2$ گیگابایت
- اندازه هر صفحه: $2^9 = 512$ بایت
- بنابراین:
- تعداد صفحات مورد نیاز: $\frac{2^{31}}{2^9} = 2^{22}$
- پس هر فرآیند نیازمند 2^{22} ورودی در جدول صفحه است.

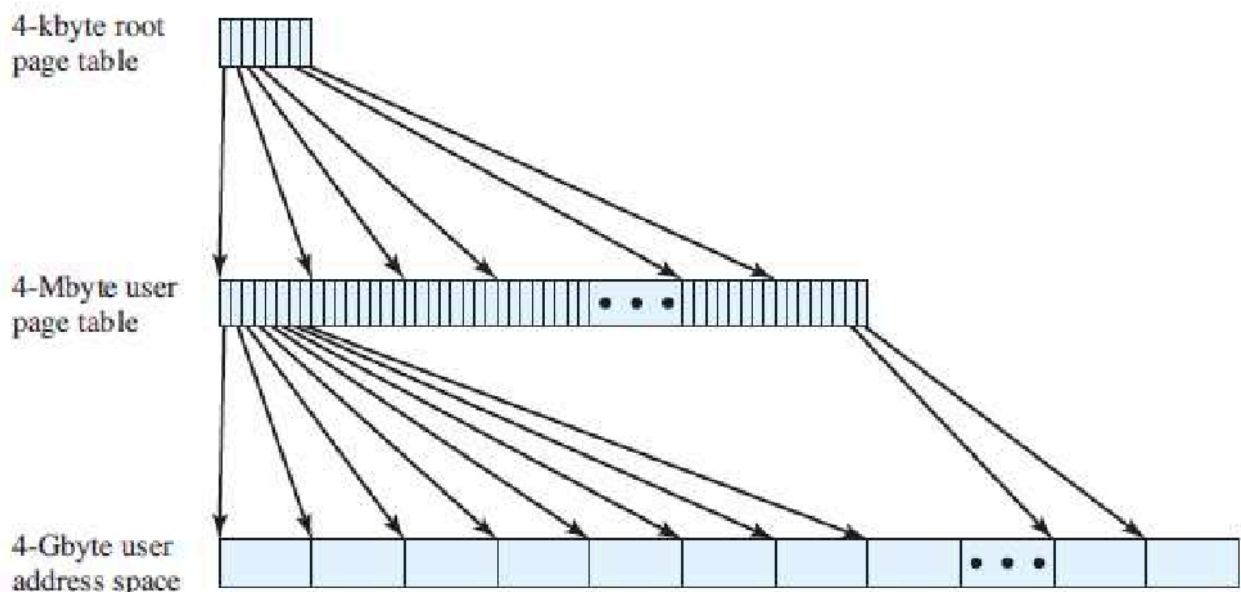
مثال ساده

فرض کنید فرآیندی 1GB فضای آدرس مجازی دارد و اندازه هر صفحه 1KB است:

- تعداد صفحات $1\text{GB} / 1\text{KB} = 1\text{M} = 2^{20}$
- پس جدول صفحه باید حداقل یک میلیون ورودی داشته باشد.

نکات کلیدی

- هر فرآیند معمولاً یک **Page Table** اختصاصی دارد.
- اندازه جدول صفحه به بزرگی فضای آدرس و اندازه صفحه وابسته است.
- جدول‌های صفحه می‌توانند بسیار بزرگ باشند (مثلاً میلیون‌ها ورودی).



A Two-Level Hierarchical Page Table (جدول صفحه) (سلسله‌مراتبی دو سطحی)

- مشکل: جدول صفحه برای هر فرآیند می‌تواند خیلی بزرگ باشد.
- راه‌حل: تقسیم جدول صفحه به چند سطح → به جای یک جدول عظیم، چند جدول کوچک‌تر و پویا.
- در این ساختار:

1. **Root Page Table** (جدول صفحه ریشه)
2. **User Page Table** (جدول صفحه سطح دوم)
3. **User Address Space** (فضای آدرس کاربر)

مراحل دسترسی

1. آدرس مجازی به چند بخش تقسیم می‌شود:

- **Root Index** (انتخاب ورودی از جدول ریشه)
- **User Page Index** (انتخاب ورودی از جدول صفحه سطح دوم)
- **Offset** (جابه‌جایی داخل صفحه)

2. ابتدا جدول ریشه → مکان جدول صفحه کاربر را نشان می‌دهد.
3. سپس جدول صفحه کاربر → مکان فریم حافظه را مشخص می‌کند.
4. در نهایت با Offset → آدرس فیزیکی ساخته می‌شود.

دیagram

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

مثال ساده

- فرض کنید آدرس مجازی 32 بیتی باشد.
- 10 بیت اول → انتخاب ورودی از Root Page Table
- 10 بیت بعدی → انتخاب ورودی از User Page Table
- 12 بیت آخر → Offset
- با این روش فقط جدول‌های لازم ساخته می‌شوند (نه همه‌ی فضای آدرس).

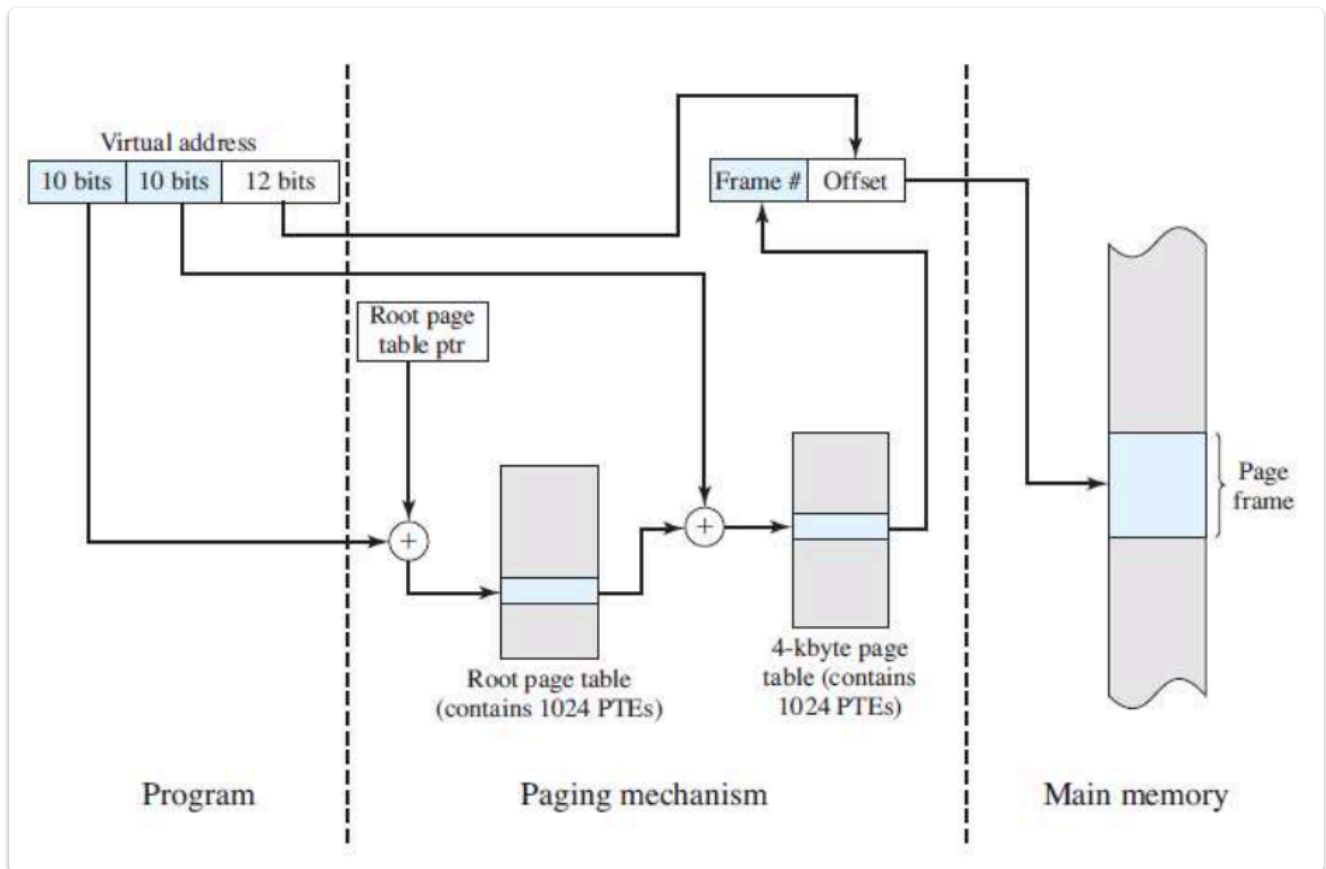
مزایا

- کاهش اندازه جدول صفحه.
- فقط بخش‌هایی از فضای آدرس که واقعاً استفاده می‌شوند، جدول صفحه دارند.

نکات کلیدی

- برای حل مشکل بزرگی جدول صفحه معرفی شده است **Hierarchical Paging**.
- ساختار دو سطحی شامل: Root Page Table + User Page Table.
- آدرس مجازی به چند بخش تقسیم می‌شود.

- مزیت: حافظه کمتری برای نگهداری جدول‌ها نیاز است.



توضیحات فراموش نشه

Inverted Page Table (IPT)

ایده اصلی

- به جای داشتن یک جدول صفحه بزرگ برای هر فرآیند، تنها **یک جدول صفحه معکوس** برای کل سیستم وجود دارد.
- در این جدول، هر **فریم فیزیکی** در حافظه‌ی اصلی دقیقاً یک ورودی دارد (نه هر صفحه مجازی).

سازوکار

1. **آدرس مجازی** به دو بخش تقسیم می‌شود:
 - شماره‌ی صفحه مجازی (Virtual Page Number)

- Offset

2. به جای جستجوی مستقیم:

- شماره صفحه مجازی → توسط **تابع هش** به یک مقدار Hash تبدیل می‌شود.

- اشاره‌گر به **جدول صفحه معکوس** → Hash

3. جدول معکوس:

- شامل اطلاعات مربوط به اینکه کدام صفحه مجازی روی کدام فریم فیزیکی قرار دارد.
- هر فریم حافظه اصلی یک ورودی دارد.

دیاگرام ساده

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

مزایا

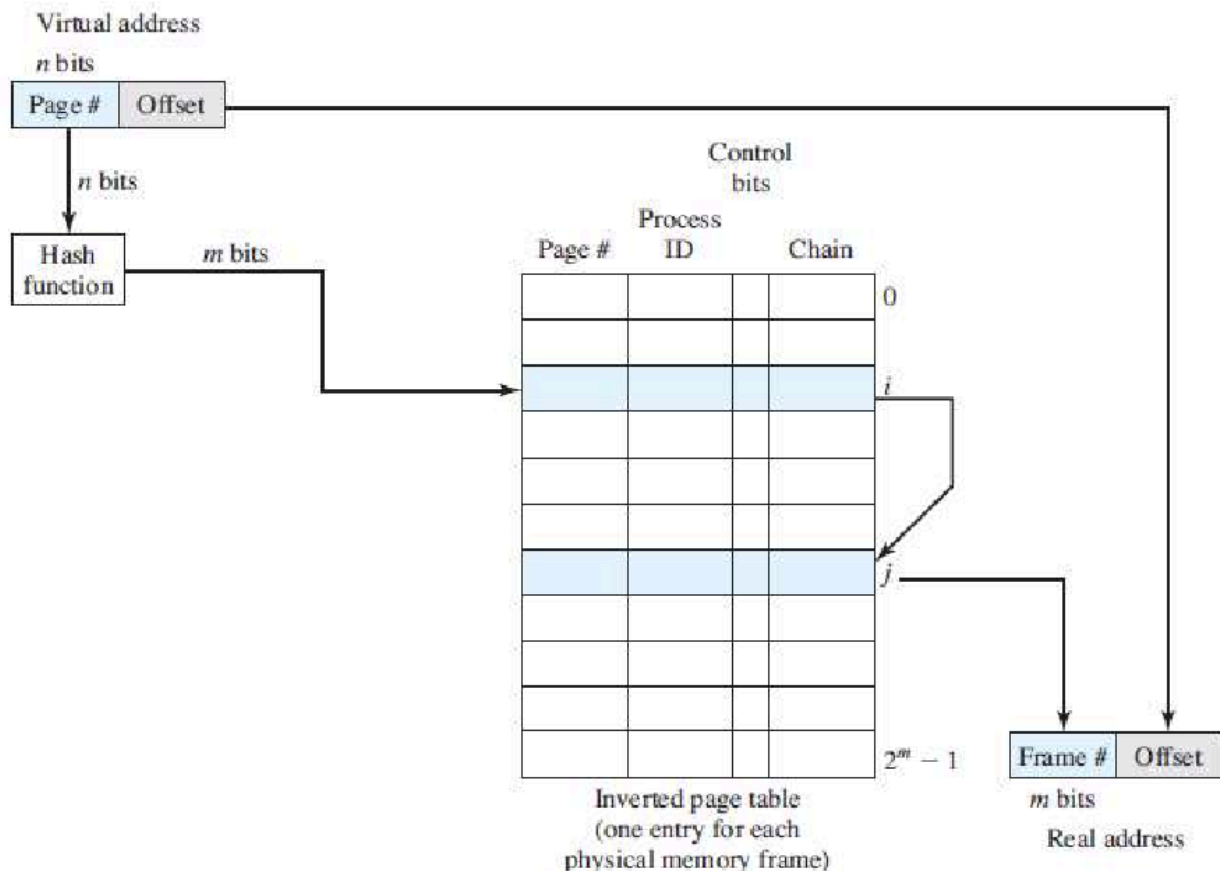
- تنها به اندازه تعداد فریم‌های حافظه اصلی ورودی نیاز دارد (ثابت).
- صرفه‌جویی قابل توجه در حافظه برای جدول‌های صفحه.
- مستقل از تعداد فرآیندها یا اندازه فضای آدرس مجازی.

معایب

- نیاز به محاسبه‌ی هش → زمان بیشتر برای آدرس‌دهی.
- امکان برخورد (Collision) در هش → نیاز به بررسی زنجیره‌ای (chaining).
- پیچیدگی بیشتر در پیاده‌سازی نسبت به جدول‌های چندسطحی.

نکته کلیدی

- یک جدول جهانی برای کل سیستم است (نه برای هر فرآیند جداگانه) IPT
- مقیاس آن بر اساس فریم‌های فیزیکی است، نه صفحات مجازی.



توضیحات لازمه

•

Translation Lookaside Buffer (TLB)

مشکل

- در سیستم حافظه مجازی، هر ارجاع به آدرس مجازی نیازمند ۲ دسترسی حافظه است:
 1. دسترسی به حافظه برای پیدا کردن Page Table Entry (PTE)
 2. دسترسی به حافظه برای آوردن داده واقعی
- این فرآیند کند است چون عملاً دو برابر زمان می‌برد.

راه حل: TLB

- برای حل مشکل، از یک **حافظه‌ی کش با سرعت بالا** استفاده می‌شود به نام **Translation Lookaside Buffer (TLB)**.
- مشابه کش پردازنده عمل می‌کند، ولی مخصوص **Page Table Entries** است.
- شامل **صفحات اخیراً استفاده‌شده** است TLB

نحوه عملکرد

1. پردازنده هنگام دسترسی به آدرس مجازی → اول TLB بررسی می‌شود.
- **TLB Hit:** (برای داده) آدرس فیزیکی بلافاصله پیدا می‌شود → تنها ۱ بار دسترسی به حافظه (برای داده).
- **TLB Miss:** بارگذاری می‌شود TLB جدید به PTE در حافظه جستجو شود → سپس Page Table باید ابتدا

دیاگرام ساده

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

مزایا

- کاهش چشمگیر زمان دسترسی به حافظه.
- بهره‌وری بیشتر نسبت به استفاده مستقیم از Page Table.
- بهره‌وری بالا مخصوصاً برای برنامه‌هایی با **locality بالا** (صفحات تکراری در مدت کوتاه).

نکته کلیدی

- یک کش سخت‌افزاری کوچک اما بسیار سریع است TLB.
- به‌طور معمول فقط چند صد ورودی دارد.

Translation Lookaside Buffer (TLB) – Workflow

مراحل دسترسی به آدرس مجازی

1. پردازنده آدرس مجازی (Virtual Address) تولید می‌کند.
2. **بررسی TLB:**
 - اگر **TLB Hit** → شماره‌ی Frame مستقیم گرفته می‌شود → آدرس واقعی ساخته می‌شود.
 - اگر **TLB Miss** → باید Page Table بررسی شود.
3. در صورت **TLB Miss:**
 - پردازنده شماره صفحه (Page Number) را در جدول صفحه (Page Table) جستجو می‌کند.
 - اگر **Present Bit = 1** → صفحه در حافظه اصلی است → Frame Number گرفته می‌شود.
 - سپس TLB با این ورودی جدید **به‌روزرسانی** می‌شود.

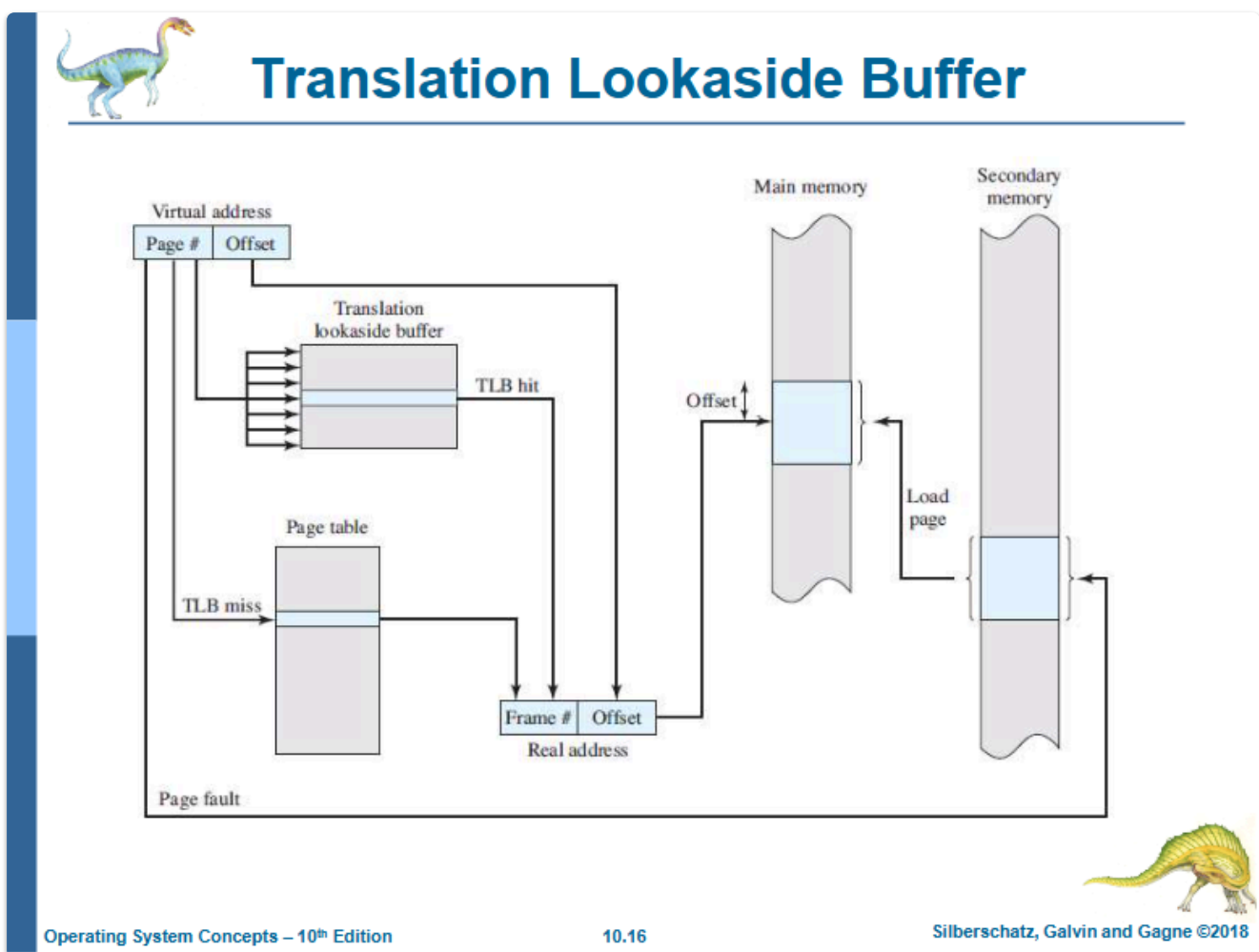
جدول مقایسه TLB Hit و TLB Miss

وضعیت	عمل انجام شده	تعداد دسترسی حافظه	توضیح
TLB Hit	گرفته می شود از TLB از Frame	1 (فقط داده)	سریع ترین حالت
TLB Miss + Page موجود در RAM	ابتدا Page Table بررسی می شود، سپس TLB آپدیت	2 (Page Table + Data)	کندتر از Hit ولی بدون Page Fault
TLB Miss + Page روی دیسک (Page Fault)	باید صفحه از دیسک به RAM آورده شود، سپس TLB آپدیت	خیلی زیاد (هزینه دیسک)	کندترین حالت

دیاگرام فرآیند

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')



خلاصه روند شکل – Translation Lookaside Buffer (TLB)

1. **Virtual Address** به دو قسمت تقسیم می شود:

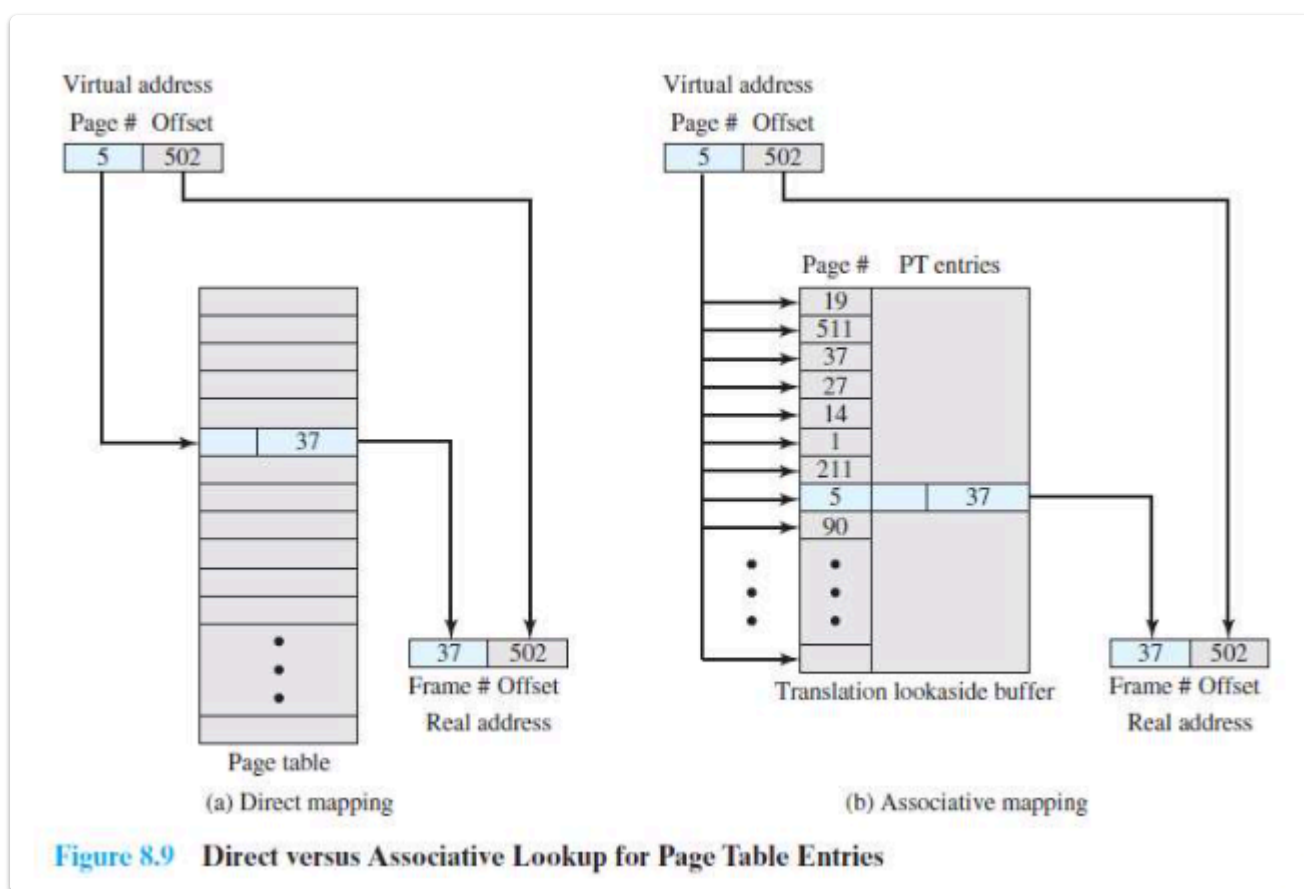
- **Page #**
- **Offset**

2. بررسی در TLB:

- اگر **TLB Hit** → مستقیماً **Frame #** پیدا میشه → آدرس واقعی ساخته میشه → داده از حافظه اصلی (Main Memory) خوانده میشه.
 - اگر **TLB Miss** → پردازنده باید **Page Table** رو نگاه کنه.
 - اگر صفحه موجود باشه (**Present Bit = 1**) → **Frame #** پیدا میشه، **TLB** آپدیت میشه.
 - اگر موجود نباشه → **Page Fault** رخ میده → صفحه از Secondary Memory (مثل دیسک) بارگذاری میشه تو **Page** RAM → **Table** و **TLB** آپدیت میشن.
3. در نهایت، **Real Address = Frame # + Offset** ساخته میشه.

این تصویر ۳ حالت اصلی رو نشون میده:

- **TLB Hit** → سریع‌ترین
- **TLB Miss** → **RAM** موجود در **Page** ولی
- **TLB Miss + Page Fault** (نیاز به دیسک) → خیلی کند



این شکل داره دو روش مختلف استفاده از **Translation Lookaside Buffer (TLB)** رو نشون میده:

1. Direct Mapping (سمت چپ)

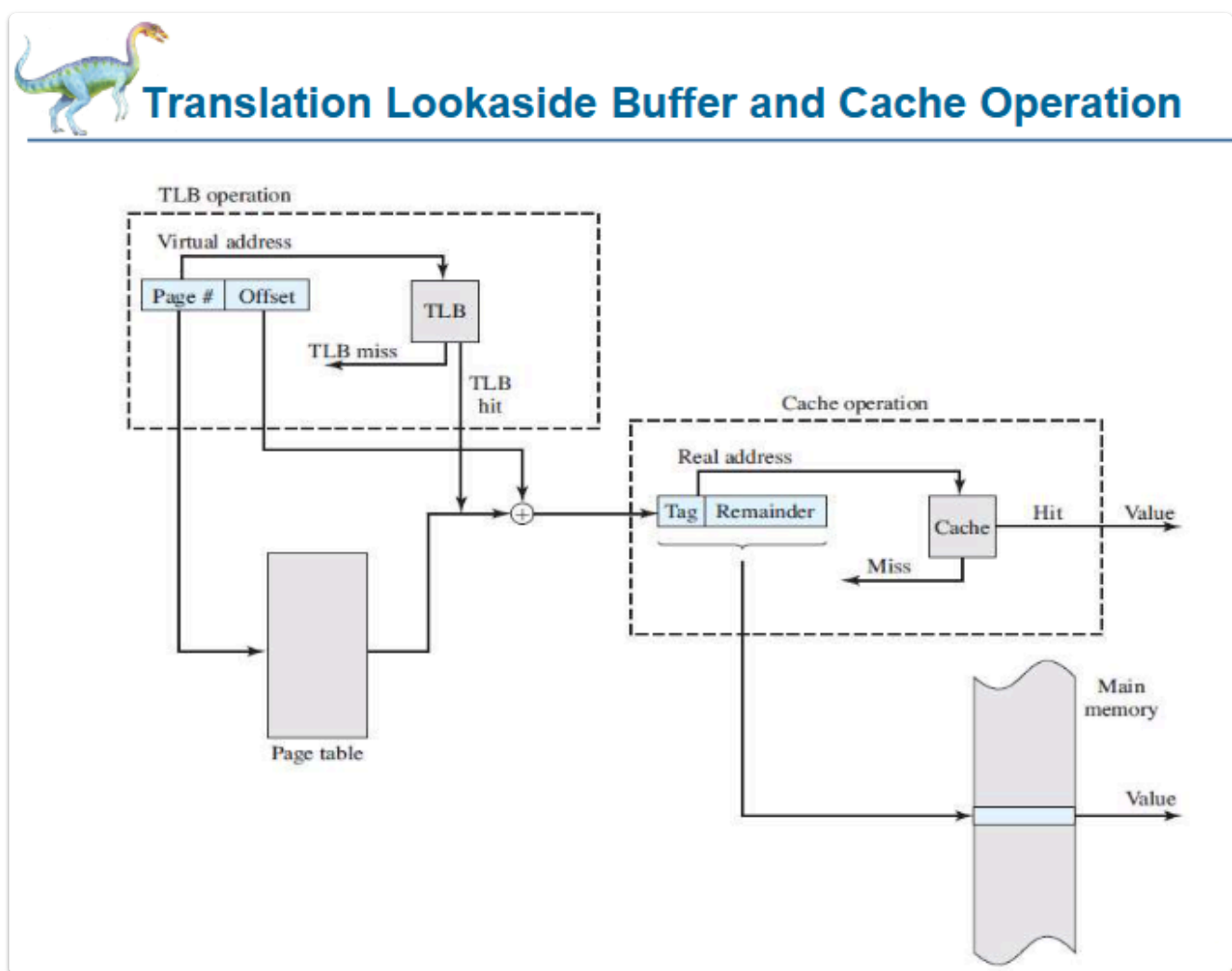
- پردازنده از **Virtual Address (Page # + Offset)** استفاده می‌کنه.
- از اونجا **Frame #** پیدا میشه.
- در نهایت، **Real Address = Frame # + Offset** ساخته میشه.
- این روش مستقیمه، اما زمان بیشتری می‌بره چون همیشه باید **Page Table** رو نگاه کنیم.

2. Associative Mapping (سمت راست)

- پردازنده باز هم **Virtual Address (Page # + Offset)** رو می‌سازه.
- همیشه **TLB (Translation Lookaside Buffer)** وارد **Page #** دارد.
- در TLB به صورت **Associative Search (جستجوی موازی)** بررسی میشه که آیا این **Page #** قبلاً ترجمه شده یا نه:
 - اگر باشه (**TLB Hit**) → سریع **Frame #** پیدا میشه → **Real Address** ساخته میشه.
 - اگر نباشه (**TLB Miss**) → باید بریم سراغ **Frame #** **Page Table** → **Page #** بیاریم → TLB آپدیت کنیم.

نتیجه: ✓

- **Direct Mapping** → کمک بگیریم Page Table کندتر، چون همیشه باید از
- **Associative Mapping (TLB)** گرفته میشه TLB از **Frame #** سریع‌تر، چون بیشتر وقت‌ها مستقیماً →



این شکل تعامل **Cache** و **TLB** رو نشون می‌ده:

1. TLB Operation (سمت چپ)

- بررسی می‌کنه **TLB**:
 - می‌گیریم **Frame #** مستقیماً → **TLB hit**

• آدرس مجازی = **Page # + Offset**.

- TLB miss → باید از Page Table اصلی در حافظه اصلی Frame # بعد رو بیاریم و بعد TLB رو آپدیت کنیم

2. Cache Operation (سمت راست)

- بعد از تشکیل Real Address (Frame # + Offset) → وارد Cache میشه.
- بررسی می‌کنه Cache:
 - Hit → داده سریعاً برگردونده میشه.
 - Miss → ذخیره میشه Cache آورده میشه و در (Main Memory) داده از حافظه اصلی → Miss

نتیجه نهایی: ✓

- رو بالا می‌بره (Virtual → Physical) سرعت ترجمه آدرس TLB
- سرعت دسترسی به داده‌های حافظه اصلی رو بالا می‌بره Cache
- این دو با هم ترکیب بشن، دسترسی حافظه خیلی سریع‌تر میشه چون هم lookup سریع‌تره هم داده‌ها در cache نزدیک CPU هستن.

در حافظه مجازی (اندازه صفحه) Page Size

توضیح ساده و قابل فهم

یکی از تصمیم‌های مهم در طراحی سخت‌افزار و سیستم عامل تعیین اندازه صفحه (Page Size) است. انتخاب اندازه صفحه بر چندین عامل اثر دارد:

1. Internal Fragmentation (قطعه‌بندی داخلی)

- صفحات کوچک‌تر → قطعه‌بندی داخلی کمتر
 - چون فضای خالی بلااستفاده درون یک صفحه کوچک‌تر خواهد بود.
 - مثال: اگر یک فرآیند به 4100 بایت نیاز داشته باشد و اندازه صفحه 4KB باشد، یک صفحه کامل (4096 بایت) + 4 بایت دیگر استفاده می‌شود. → 4092 بایت هدر می‌رود.
- صفحات بزرگ‌تر → قطعه‌بندی داخلی بیشتر
 - فضای خالی بیشتری هدر می‌رود چون ممکن است آخرین صفحه فرآیند به‌طور کامل پر نشود.

2. تعداد صفحات و اندازه جدول صفحات

- صفحات کوچک‌تر → تعداد صفحات بیشتر برای هر فرآیند.
- این یعنی جدول صفحات بزرگ‌تر می‌شود و فضای بیشتری از حافظه اصلی باید صرف نگهداری Page Table شود.
- اگر جدول صفحات خیلی بزرگ باشد، بخشی از آن باید در حافظه مجازی نگهداری شود.
- در این حالت، ممکن است یک دسترسی ساده به حافظه باعث دو Page Fault شود:
 1. یکی برای پیدا کردن ورودی جدول صفحه.
 2. دیگری برای دسترسی به خود صفحه مورد نظر.

3. ویژگی سخت‌افزار ذخیره‌سازی ثانویه (Secondary Storage)

- بیشتر دستگاه‌های ذخیره‌سازی (مثل دیسک سخت سنتی) **چرخشی هستند** و از بلوک‌های بزرگ داده برای انتقال بهینه استفاده می‌کنند.
- بنابراین **صفحات بزرگ‌تر** برای **انتقال بلوک‌های بزرگ داده** کارایی بیشتری دارند.

مثال آموزشی ساده

- اگر اندازه صفحه **1KB** باشد و فرآیندی به **8MB** حافظه نیاز داشته باشد → به **8192 صفحه** نیاز است.
 - اما اگر اندازه صفحه **4KB** باشد → فقط به **2048 صفحه** نیاز است.
 - در حالت 1KB، جدول صفحات 4 برابر بزرگ‌تر خواهد بود.
- این نشان می‌دهد که انتخاب اندازه صفحه یک **تبادل (Trade-off)** بین کاهش قطعه‌بندی داخلی و افزایش کارایی مدیریت جدول صفحات است.

نمودار Mermaid: تأثیر اندازه صفحه

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- انتخاب اندازه صفحه (Page Size) یک تصمیم حیاتی در طراحی سیستم است.
- **صفحات کوچک‌تر:** قطعه‌بندی داخلی کمتر، اما جدول صفحات بزرگ‌تر.
- **صفحات بزرگ‌تر:** جدول صفحات کوچک‌تر و کارایی I/O بیشتر، اما قطعه‌بندی داخلی بیشتر.
- در سیستم‌های واقعی، اندازه صفحه معمولاً یک **توازن (Trade-off)** بین این عوامل است (مثلاً 4KB یا 8KB).
- اندازه صفحه تأثیر مستقیمی بر **کارایی حافظه مجازی** دارد.

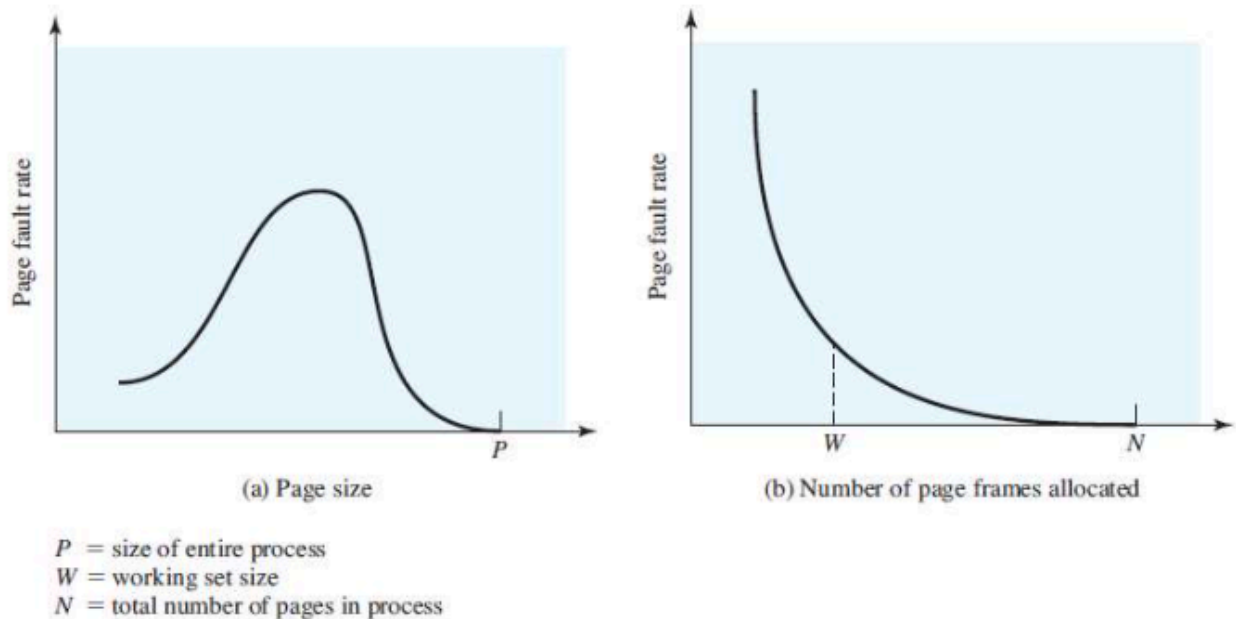


Figure 8.11 Typical Paging Behavior of a Program

رفتار معمولی صفحه‌بندی یک برنامه (Typical Paging Behavior)

(a) Page Size در برابر اندازه صفحه Page Fault نمودار سمت چپ - نرخ

- محور افقی: اندازه صفحه (Page Size)
- محور عمودی: نرخ Page Fault

تحلیل:

1. در ابتدا، وقتی اندازه صفحه خیلی کوچک است:

- بسیار بزرگ می‌شود Page Table.
- احتمال وقوع Double Page Fault افزایش می‌یابد.
- به همین دلیل، نرخ Page Fault بالاست.
- 2. وقتی اندازه صفحه کمی افزایش می‌یابد:
- سیستم تعادلی بین هزینه جدول صفحات و قطعه‌بندی داخلی برقرار می‌کند.
- نرخ Page Fault کاهش پیدا می‌کند.
- 3. اگر اندازه صفحه بیش از حد بزرگ شود:
- قطعه‌بندی داخلی (Internal Fragmentation) شدید می‌شود.
- نرخ Page Fault دوباره افزایش می‌یابد.

→ بنابراین یک اندازه بهینه صفحه (Optimal Page Size) وجود دارد که نرخ Page Fault را حداقل می‌کند.

(b) در برابر تعداد قاب‌های اختصاص داده شده Page Fault نمودار سمت راست - نرخ

- محور افقی: تعداد قاب‌های اختصاص داده شده به فرآیند (از N تا ∞ = تعداد کل صفحات فرآیند).
- محور عمودی: نرخ Page Fault

تحلیل:

1. اگر تعداد قاب‌ها خیلی کم باشد:

- کوچک است Resident Set.
- نرخ Page Fault خیلی زیاد است (چون فرآیند مدام نیاز به صفحات جدید دارد).

2. وقتی تعداد قاب‌ها افزایش یابد:
- نرخ Page Fault کاهش شدیدی پیدا می‌کند.
3. در نقطه‌ای که برابر با **Working Set Size (W)** است:
- فرآیند تقریباً همه صفحات مورد نیاز خود را در حافظه دارد.
 - از اینجا به بعد، افزودن قاب‌های بیشتر تأثیر کمی دارد.

مثال آموزشی ساده

فرض کنید یک فرآیند شامل ۱۰۰ صفحه است و اندازه Working Set آن ۲۰ صفحه است:

- اگر فقط ۵ قاب به فرآیند بدهیم → نرخ Page Fault بسیار زیاد می‌شود (Thrashing).
- اگر ۲۰ قاب بدهیم → فرآیند تقریباً بدون Page Fault اجرا می‌شود.
- اگر ۴۰ قاب بدهیم → تفاوت زیادی با حالت ۲۰ قاب ندارد، چون بیش از نیاز فرآیند حافظه تخصیص داده‌ایم.

نکات کلیدی

- **Page Size:**
 - کوچک‌تر → Page Table بزرگ‌تر، Double Page Fault
 - بزرگ‌تر → Internal Fragmentation بیشتر
 - یک اندازه بهینه وجود دارد.
- **تعداد قاب‌ها (Page Frames):**
 - کمتر از Working Set → نرخ Page Fault بالا، احتمال Thrashing
 - برابر یا بیشتر از Working Set → نرخ Page Fault خیلی کم
 - بیشتر از Working Set → بهبود قابل توجهی ندارد.

Segmentation (بخش‌بندی حافظه)

توضیح ساده و قابل فهم

تقسیم می‌شود. هر بخش می‌تواند اندازه (Segment) یک روش مدیریت حافظه است که در آن حافظه به چندین **بخش Segmentation** متفاوتی داشته باشد و حتی اندازه‌اش در زمان اجرا تغییر کند.

1. ویژگی اصلی Segmentation

- حافظه نه به صورت صفحات ثابت، بلکه به صورت **بخش‌های منطقی** دیده می‌شود.
- هر بخش می‌تواند مربوط به یک **ماژول برنامه، تابع، آرایه یا داده خاص** باشد.
- اندازه بخش‌ها می‌تواند **نامساوی و پویا** باشد.

2. مزایای Segmentation

- **مدیریت راحت‌تر ساختارهای داده در حال رشد:** مثلاً یک آرایه که در طول اجرای برنامه بزرگ‌تر می‌شود می‌تواند در یک Segment مجزا قرار گیرد.
- **انعطاف در توسعه و تغییر برنامه:** اگر بخشی از برنامه تغییر کند، فقط همان Segment تغییر می‌کند و نیازی به بارگذاری مجدد کل برنامه نیست.
- **اشتراک‌گذاری میان فرآیندها:** چندین فرآیند می‌توانند یک Segment (مثل کتابخانه مشترک) را استفاده کنند.
- **امکان حفاظت (Protection):** هر Segment می‌تواند مجوز دسترسی مخصوص خودش را داشته باشد (Read/Write/Execute).

مثال آموزشی ساده

فرض کنید برنامه‌ای شامل سه بخش است:

- کد (Code Segment): شامل دستورالعمل‌های برنامه.
- داده (Data Segment): شامل متغیرها و آرایه‌ها.
- پشته (Stack Segment): شامل فراخوانی توابع و متغیرهای محلی.

هر کدام از این بخش‌ها در حافظه به صورت یک Segment مجزا نگهداری می‌شوند و می‌توانند به طور مستقل رشد کنند یا تغییر کنند.

نکات کلیدی

- حافظه را به بخش‌های منطقی تقسیم می‌کند (کد، داده، پشته و ...) **Segmentation**.
- بخش‌ها می‌توانند **اندازه متفاوت و پویا** داشته باشند.
- مزایا:
 - مدیریت ساده ساختارهای در حال رشد.
 - امکان تغییر و بازکامپایل بخشی از برنامه بدون تأثیر روی کل.
 - قابلیت اشتراک‌گذاری بین فرآیندها.
 - اعمال حفاظت (Permissions) بر اساس Segment.

Segmentation Table (جدول بخش‌بندی حافظه)

توضیح ساده و قابل فهم

برای مدیریت **Segmentation**، سیستم عامل از یک **Segment Table** استفاده می‌کند. این جدول برای هر فرآیند ایجاد می‌شود و اطلاعات مربوط به بخش‌های مختلف آن فرآیند را نگهداری می‌کند.

هر ورودی (Entry) در جدول بخش‌بندی شامل موارد زیر است:

- Starting Address (آدرس شروع بخش در حافظه اصلی)**
 - نشان می‌دهد که بخش مربوطه از کدام آدرس در حافظه اصلی آغاز می‌شود.
- Length (طول بخش)**
 - مشخص می‌کند که بخش چه اندازه‌ای دارد.
 - این ویژگی باعث می‌شود که حافظه بتواند بخش‌هایی با اندازه‌های متغیر و پویا را مدیریت کند.
- Present Bit (بیت حضور)**
 - مشخص می‌کند که آیا این بخش در حال حاضر در حافظه اصلی وجود دارد یا نه.
 - اگر نباشد و پردازنده به آن دسترسی بخواهد → **Segment Fault** رخ می‌دهد و سیستم عامل باید آن را از دیسک بارگذاری کند.
- Modify Bit (بیت تغییر)**
 - مشخص می‌کند که آیا محتویات بخش از آخرین باری که به حافظه اصلی بارگذاری شده، تغییر کرده است یا نه.
 - اگر تغییر کرده باشد و بخواهیم آن را از حافظه خارج کنیم، باید قبلش روی دیسک ذخیره شود (Write Back).

مثال آموزشی ساده

فرض کنید یک فرآیند شامل ۳ بخش است:

Segment #	Starting Address	Length	Present (P)	Modified (M)
0 (Code)	1000	2000	1	0
1 (Data)	5000	1500	1	1

Segment #	Starting Address	Length	Present (P)	Modified (M)
2 (Stack)	–	1000	0	0

- بخش **Code** در حافظه اصلی است (P=1) و تغییری نکرده (M=0).
- بخش **Data** در حافظه اصلی است (P=1) و تغییر کرده است (M=1) → اگر بخواهیم آن را خارج کنیم، باید روی دیسک ذخیره شود.
- بخش **Stack** در حافظه اصلی نیست (P=0) → اگر CPU به آن دسترسی بخواهد، **Segment Fault** رخ می‌دهد.

نمودار Mermaid: ساختار جدول بخش‌بندی

!Error parsing Mermaid diagram

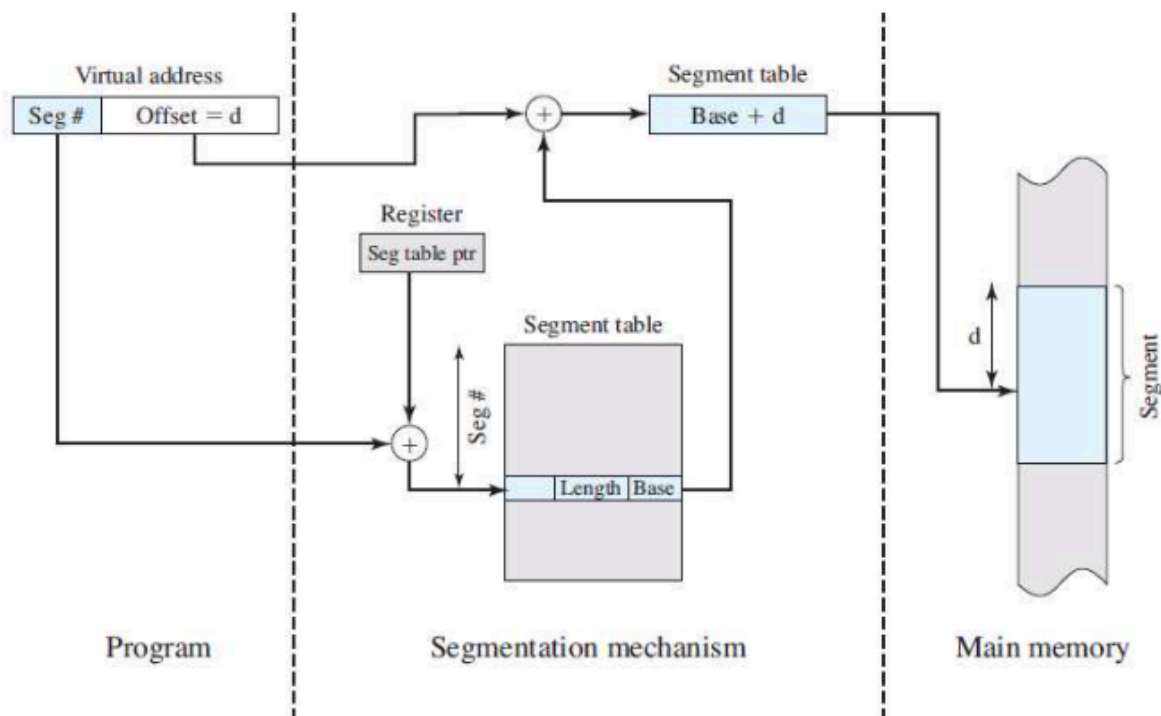
Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- برای هر فرآیند ایجاد می‌شود **Segment Table**.
- هر ورودی شامل:
 - آدرس شروع بخش
 - طول بخش
 - بیت حضور (P) → نشان می‌دهد بخش در حافظه است یا نه
 - بیت تغییر (M) → نشان می‌دهد بخش تغییر کرده یا نه
- اگر P=0 و CPU به آن بخش نیاز داشته باشد → **Segment Fault** رخ می‌دهد.
- اگر M=1 و بخواهیم بخش را خارج کنیم → باید ابتدا روی دیسک ذخیره شود.



Address Translation



Address Translation در Segmentation

توضیح ساده و قابل فهم

1. آدرس مجازی (Virtual Address)

- هر آدرس مجازی از دو بخش تشکیل می‌شود:

- **Segment Number** (شماره بخش)
- **Offset** (d = جابجایی داخل بخش)

مثال: اگر برنامه بخواهد به متغیر داخل بخش داده (Data Segment) دسترسی پیدا کند، آدرس آن به صورت $(Seg\#, d)$ بیان می‌شود.

2. Segmentation Table

- پردازنده ابتدا از **Segment Number** استفاده می‌کند تا ورودی متناظر در جدول بخش‌ها (Segment Table) را پیدا کند.
- هر ورودی شامل:
- **Base** (آدرس شروع بخش در حافظه اصلی)
- **Length** (طول بخش)

3. بررسی اعتبار آدرس

- اگر $Offset (d) > Length$ باشد $\rightarrow$ دسترسی غیرمجاز است و خطای **Segmentation Fault** رخ می‌دهد.
- در غیر این صورت، دسترسی معتبر است.

4. محاسبه آدرس فیزیکی

- Physical Address = Base + Offset

• بدین ترتیب آدرس فیزیکی به دست می‌آید و از حافظه اصلی بازیابی می‌شود.

مثال آموزشی ساده

فرض کنید:

- Segment #2 دارای Base = 4000 و Length = 1000 است.
- برنامه می‌خواهد به آدرس (Seg=2, d=200) دسترسی پیدا کند.
- محاسبه:
- $Physical\ Address = 4000 + 200 = 4200$
- چون $Length(1000) > 200$ ، دسترسی معتبر است.
- اما اگر (Seg=2, d=1500) بود:
- چون $Length(1000) < 1500$ ، خطای Segmentation Fault رخ می‌دهد.

نکات کلیدی

- آدرس مجازی = (Segment Number, Offset)
- جدول بخش‌ها شامل Base و Length هر بخش است.
- اگر $Offset > Length \rightarrow$ Segmentation Fault
- اگر معتبر باشد:
- آدرس فیزیکی = $Base + Offset$
- دسترس به حافظه اصلی انجام می‌شود.

Combined Paging and Segmentation

توضیح ساده و قابل فهم

هرکدام از دو روش Paging و Segmentation مزایا و معایب خاص خودشان را دارند:

1. Paging (صفحه‌بندی)

- برای برنامه‌نویس شفاف است (یعنی برنامه‌نویس نیازی ندارد به آن فکر کند).
- مشکل external fragmentation (تکه‌تکه شدن خارجی) را برطرف می‌کند.
- استفاده‌ی کارآمدتری از حافظه اصلی می‌دهد.

2. Segmentation (بخش‌بندی)

- برای برنامه‌نویس قابل مشاهده است (برنامه‌نویس می‌تواند بخش‌های مختلف مثل Code, Data, Stack را ببیند).
- مدیریت راحت‌تر ساختارهای داده‌ای در حال رشد (مثل آرایه‌ها).
- پشتیبانی از modularity (ماژولار بودن برنامه).
- امکان اشتراک‌گذاری (Sharing) و حفاظت (Protection) در سطح بخش‌ها.

ایده‌ی ترکیب

برای این که هم از مزایای **Paging** و هم از مزایای **Segmentation** استفاده کنیم، برخی سیستم‌ها سخت‌افزار پردازنده و نرم‌افزار سیستم‌عامل را طوری طراحی می‌کنند که هر دو مکانیزم با هم ترکیب شوند:

- هر فرآیند ابتدا به صورت **Segments** تقسیم می‌شود (مطابق نیاز منطقی برنامه).
- سپس هر Segment به **Pages** تقسیم می‌شود تا مشکل تکه‌تکه شدن خارجی (external fragmentation) حل شود.
- در نتیجه:
- برنامه‌نویس هنوز می‌تواند با بخش‌ها (Segments) کار کند.
- سیستم‌عامل حافظه را به صورت صفحات مدیریت می‌کند و از مزایای Paging بهره می‌برد.

مثال آموزشی ساده

فرض کنید یک برنامه سه بخش دارد:

- Code (برنامه)
- Data (داده‌ها)
- Stack (پشته)

در سیستم ترکیبی:

- هر بخش (Segment) جداگانه تعریف می‌شود.
- سپس هر بخش به صفحات کوچکتر (Pages) تقسیم شده و در حافظه قرار می‌گیرد.

نمودار Mermaid: ترکیب Paging و Segmentation

!Error parsing Mermaid diagram

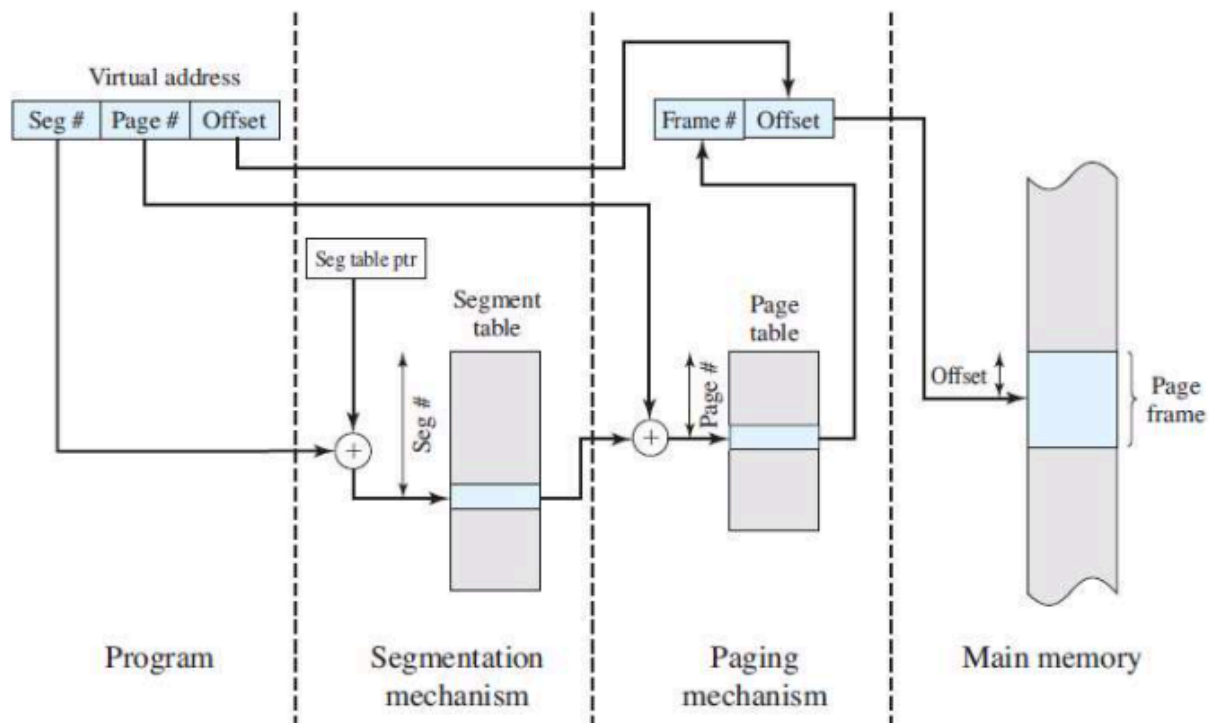
Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- **Paging** → کارایی حافظه بالا، external fragmentation شفاف برای برنامه‌نویس، حذف.
- **Segmentation** → قابل مشاهده برای برنامه‌نویس، پشتیبانی از رشد داده‌ها، ماژولار بودن، حفاظت و اشتراک‌گذاری.
- **ترکیب Paging + Segmentation:**
 - برنامه به بخش‌ها تقسیم می‌شود.
 - هر بخش به صفحات تقسیم می‌شود.
 - بهترین ویژگی‌های هر دو روش با هم به دست می‌آید.



Combined Paging and Segmentation



Combined Paging and Segmentation – Address Translation

توضیح ساده و قابل فهم

در این مکانیزم، یک آدرس مجازی (Virtual Address) شامل سه بخش است:

1. Segment Number (شماره بخش)
2. Page Number (شماره صفحه در آن بخش)
3. Offset (جابجایی در صفحه)

مراحل ترجمه آدرس:

1. مرحله Segmentation

- ابتدا CPU از Segment Number استفاده می‌کند تا به Segment Table دسترسی پیدا کند.
- هر ورودی جدول بخش‌ها شامل اطلاعاتی مثل Base Address برای جدول صفحات آن بخش است.
- با این کار، می‌فهمیم جدول صفحات (Page Table) مربوط به این بخش کجاست.

2. مرحله Paging

- سپس از Page Number برای جستجو در Page Table استفاده می‌شود.
- این جدول مشخص می‌کند که صفحه مورد نظر در کدام Frame از حافظه اصلی قرار دارد.

3. تشکیل آدرس فیزیکی

- $\text{Physical Address} = \text{Frame Number} + \text{Offset}$

- حالا پردازنده می‌تواند به حافظه اصلی دسترسی پیدا کند.

مثال آموزشی ساده

فرض کنید آدرس مجازی به شکل زیر باشد:

Seg = 2, Page = 5, Offset = 100

- در **Segment Table**، ورودی شماره 2 را پیدا می‌کنیم → این ورودی ما را به **Page Table** مربوط به بخش 2 هدایت می‌کند.
- در **Page Table** بخش 2، ورودی شماره 5 را پیدا می‌کنیم → فرض کنید Frame = 900.
- حالا آدرس فیزیکی می‌شود:

$$1000 = 100 + 900$$

- یعنی داده مورد نظر در آدرس فیزیکی 1000 در حافظه اصلی قرار دارد.

نمودار Mermaid: روند ترجمه آدرس

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- آدرس مجازی در این مدل (Seg#, Page#, Offset) →
 - به ما می‌گوید جدول صفحات مربوط به هر بخش کجاست → **Segmentation**
 - در حافظه اصلی نگاشت می‌کند Frame مورد نظر را به → **Paging**
- در نهایت → **Physical Address = Frame + Offset**
- ترکیب این دو روش:
 - مزایای **Segmentation** (ماژولار بودن، حفاظت، اشتراک‌گذاری)
 - مزایای **Paging** (حذف External Fragmentation، مدیریت کارآمد حافظه)

Operating System Software – Memory Management Design

خلاصه:

- طراحی بخش مدیریت حافظه در سیستم‌عامل به سه تصمیم کلیدی بستگی دارد:
۱. استفاده یا عدم استفاده از حافظه مجازی،
 ۲. انتخاب بین صفحه‌بندی (Paging)، قطعه‌بندی (Segmentation) یا ترکیب آن‌ها،
 ۳. الگوریتم‌های مدیریت حافظه (مثل جایگزینی صفحه).

توضیح کامل:

مدیریت حافظه یکی از مهم‌ترین وظایف سیستم‌عامل است چون تعیین می‌کند فرآیندها چطور از فضای حافظه استفاده کنند. طراحی این بخش به سه انتخاب اصلی بستگی دارد:

۱. استفاده از حافظه مجازی (Virtual Memory):

- در حافظه مجازی، فرآیندها می‌توانند فضایی بزرگ‌تر از حافظه فیزیکی درخواست کنند.
- سیستم‌عامل با استفاده از دیسک (swap space) بخشی از داده‌ها را موقتاً بیرون می‌برد.

- اگر از حافظه مجازی استفاده نشود، فرآیندها فقط به اندازه حافظه فیزیکی می‌توانند کار کنند.

2. روش تخصیص حافظه – Paging یا Segmentation یا هر دو:

- **Paging (صفحه‌بندی):** حافظه به بلوک‌های ثابت تقسیم می‌شود. سادگی در مدیریت و عدم تکه‌تکه شدن خارجی (frames/صفحه‌ها) حافظه به بلوک‌های ثابت تقسیم می‌شود. از مزایای آن است (External Fragmentation).
- **Segmentation (قطعه‌بندی):** حافظه بر اساس منطق برنامه (کد، داده، استک و...) به بخش‌هایی با اندازه متغیر تقسیم می‌شود. راحتی در مدیریت منطقی برنامه مزیت آن است.
- **Paging + Segmentation:** و هم از انعطاف Paging بسیاری از سیستم‌ها ترکیبی از هر دو را به کار می‌برند تا هم از سادگی Segmentation بهره ببرند.

3. انتخاب الگوریتم‌ها:

- برای مدیریت حافظه مجازی باید الگوریتم‌هایی برای **جایگزینی صفحه** (Page Replacement) انتخاب شود، مثل LRU، FIFO، یا Clock.
- همچنین الگوریتم‌هایی برای تخصیص و آزادسازی حافظه مورد نیاز است.
- انتخاب درست الگوریتم‌ها روی کارایی و سرعت سیستم عامل بسیار اثرگذار است.

مثال ساده آموزشی:

فرض کن کامپیوتری RAM ۸GB دارد ولی یک برنامه ۱۲GB حافظه می‌خواهد.

- اگر **حافظه مجازی** فعال باشد، سیستم عامل ۸GB را از RAM می‌دهد و ۴GB باقی‌مانده را روی دیسک قرار می‌دهد.
- برای مدیریت آن، سیستم می‌تواند از **Paging** استفاده کند (تقسیم کل حافظه به بلوک‌های ۴KB) یا از **Segmentation** (کد، داده و استک هر کدام یک بخش جدا).
- اگر برنامه بیش از حد بین RAM و دیسک جابجا شود، پدیده‌ای به نام **Thrashing** رخ می‌دهد.

نمودار (Mermaid) – سه انتخاب اصلی:

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی:

- طراحی مدیریت حافظه سه انتخاب محوری دارد: حافظه مجازی، روش تخصیص (Paging/Segmentation)، الگوریتم‌ها.
- **Paging** = بلوک‌های ثابت، **Segmentation** = بلوک‌های متغیر.
- حافظه مجازی امکان اجرای برنامه‌های بزرگ‌تر از RAM را فراهم می‌کند.
- انتخاب الگوریتم‌های جایگزینی صفحه روی کارایی سیستم بسیار حیاتی است.

Operating System Policies for Virtual Memory

خلاصه:

سیاست‌های سیستم عامل برای حافظه مجازی شامل تصمیماتی در مورد نحوه بارگذاری صفحات (Fetch Policy)، انتخاب صفحه‌ها برای جایگزینی (Placement/Replacement Policy)، مدیریت مجموعه مقیم (Resident Set Management)، پاک‌سازی (Cleaning Policy) و کنترل بار (Load Control) است.

توضیح کامل:

1. Fetch Policy (سیاست واکشی صفحه)

- **Demand** (درخواست): (رخ دهد Page Fault) صفحه فقط زمانی به حافظه آورده می‌شود که واقعاً مورد نیاز شود.
 - **Prepaging** (پیش‌واکشی): (برای) چند صفحه دیگر علاوه بر صفحه مورد نیاز بارگذاری می‌شوند، به امید اینکه در آینده نزدیک استفاده شوند (Page Faultها کاهش تعداد).
-

2. Placement Policy (سیاست قرارگیری)

- تعیین می‌کند صفحه جدید در کدام بخش از حافظه قرار گیرد.
 - در سیستم‌های Paging معمولاً ساده است (هر Frame آزاد می‌تواند استفاده شود).
 - در Segmentation مهم‌تر است چون اندازه قطعات متغیر است.
-

3. Replacement Policy (سیاست جایگزینی صفحه)

وقتی حافظه پر است و صفحه جدید نیاز داریم، باید یک صفحه موجود جایگزین شود. الگوریتم‌ها:

- **Optimal**: کم‌مصرف‌ترین و ایده‌آل؛ صفحه‌ای که در دورترین آینده استفاده خواهد شد جایگزین می‌شود (عملی نیست چون نیاز به آینده‌نگری دارد).
 - **LRU (Least Recently Used)**: صفحه‌ای که طولانی‌ترین زمان استفاده نشده است جایگزین می‌شود.
 - **FIFO (First-In, First-Out)**: صفحه‌ای که زودتر از همه وارد حافظه شده بود، حذف می‌شود.
 - **Clock Algorithm**: (reference bit) با استفاده از بیت استفاده FIFO نسخه‌ای بهینه‌تر از.
 - **Page Buffering**: چند صفحه اضافی نگه‌داری می‌شود تا احتمال خطا کاهش یابد.
-

4. Resident Set Management (مدیریت مجموعه مقیم)

- **Resident Set Size**: تعداد صفحاتی که یک فرآیند در حافظه دارد.
 - **Fixed**: اندازه ثابت است.
 - **Variable**: اندازه می‌تواند تغییر کند.
 - **Replacement Scope**: محدوده انتخاب صفحه برای جایگزینی.
 - **Global**: می‌توان صفحات هر فرآیند را جایگزین کرد.
 - **Local**: فقط صفحات همان فرآیند جایگزین می‌شوند.
-

5. Cleaning Policy (سیاست پاک‌سازی)

- **Demand Cleaning**: صفحه فقط زمانی نوشته می‌شود (به دیسک) که باید جایگزین شود.
 - **Precleaning**: سیستم‌عامل زودتر صفحات تغییر یافته را به دیسک می‌نویسد تا در مواقع اضطراری آماده باشد.
-

6. Load Control (کنترل بار)

- تعداد فرآیندهایی که همزمان در حافظه هستند (degree of multiprogramming) باید مدیریت شود.
- اگر فرآیندهای زیادی در حافظه باشند → Page Fault زیاد و **Thrashing** رخ می‌دهد.
- اگر خیلی کم باشند → استفاده از CPU کاهش می‌یابد.

مثال ساده آموزشی:

فرض کن سه برنامه در حال اجرا هستند و حافظه فقط ۵ Frame دارد.

- اگر **FIFO** استفاده شود و برنامه مرتباً صفحات جدیدی نیاز داشته باشد، ممکن است صفحه‌ای که هنوز مورد نیاز است زود حذف شود.
- اگر **LRU** استفاده شود، صفحه‌ای که مدت‌ها استفاده نشده حذف می‌شود (کارآمدتر).
- با **Prepaging**، وقتی برنامه به صفحه ۵ نیاز دارد، سیستم صفحه ۶ را هم بارگذاری می‌کند چون احتمالاً بعداً استفاده می‌شود.
- در مدیریت **Global Replacement**، یک فرآیند می‌تواند صفحه‌های فرآیند دیگر را بگیرد؛ ولی در **Local Replacement** فقط صفحه‌های خودش.

نمودار (Mermaid) – سیاست‌های حافظه مجازی:

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getComputedTextLength')

نکات کلیدی:

- Fetch Policy مشخص می‌کند چه زمانی صفحه بارگذاری شود.
- Replacement Policy (FIFO, LRU, Clock) قلب عملکرد حافظه مجازی است.
- Resident Set Management تعیین می‌کند هر فرآیند چند صفحه داشته باشد.
- Cleaning Policy مشخص می‌کند صفحات تغییر یافته چه زمانی روی دیسک نوشته شوند.
- Load Control از Thrashing جلوگیری می‌کند.

Fetch Policy (سیاست واکشی صفحه)

خلاصه:

سیاست Fetch تعیین می‌کند که چه زمانی یک صفحه باید به حافظه اصلی آورده شود. دو روش رایج آن عبارتند از **Demand Paging** (واکشی هنگام نیاز) و **Prepaging** (واکشی پیش‌دستانه).

توضیح کامل:

1. تعریف

بخشی از سیاست‌های حافظه مجازی است که مشخص می‌کند چه زمانی یک صفحه از دیسک به حافظه اصلی منتقل شود Fetch Policy.

2. روش‌ها

- **Demand Paging (واکشی بر اساس نیاز):**
 - صفحه فقط وقتی به حافظه آورده می‌شود که اولین بار به آن ارجاع داده شود.
 - این کار باعث می‌شود در ابتدای اجرای یک برنامه تعداد زیادی Page Fault ایجاد شود.
 - مزیت: حافظه فقط برای صفحات واقعاً مورد نیاز استفاده می‌شود.
 - عیب: ممکن است در شروع کار، اجرای فرآیند به شدت کند شود چون باید چندین بار منتظر Page Fault باشیم.
- **Prepaging (پیش‌واکشی):**
 - علاوه بر صفحه‌ای که باعث Page Fault شده، تعدادی صفحه دیگر نیز بارگذاری می‌شوند.

- هدف: کاهش تعداد Page Fault های آینده.
- مزیت: اجرای روان تر برنامه در ادامه.
- عیب: اگر صفحات پیش واکشی شده استفاده نشوند، باعث هدر رفت حافظه و زمان I/O می شود.

مثال ساده آموزشی:

فرض کن برنامه ای می خواهد از آرایه ای بزرگ (۱۰۰ عنصر) استفاده کند.

- در **Demand Paging**: ابتدا فقط صفحه مربوط به عنصر اول بارگذاری می شود. وقتی به عنصر دوم برسیم، Page Fault رخ می دهد و صفحه دوم بارگذاری می شود؛ این روند ادامه دارد → تعداد زیادی Page Fault.
- در **Prepaging**: وقتی عنصر اول باعث Page Fault شد، سیستم علاوه بر صفحه اول، چند صفحه بعدی (مثلاً ۴ صفحه بعد) را هم بارگذاری می کند. بنابراین وقتی برنامه به عنصر دوم یا سوم رسید، دیگر Page Fault رخ نمی دهد.

نمودار (Mermaid) – مقایسه Demand و Prepaging:

!Error parsing Mermaid diagram

Cannot read properties of undefined (reading 'getBBox')

نکات کلیدی:

- Fetch Policy = تعیین زمان بارگذاری صفحه به حافظه.
- Demand Paging → زیاد در شروع Page Fault فقط هنگام نیاز، اما.
- Prepaging → اما احتمال هدررفت منابع، Page Fault پیش واکشی چند صفحه، کاهش → انتخاب سیاست مناسب به رفتار برنامه و طراحی سیستم عامل بستگی دارد.

سیاست قرارگیری صفحه/قطعه در حافظه (Placement Policy)

خلاصه:

مشخص می کند یک قطعه یا صفحه از فرآیند دقیقاً در کجای حافظه اصلی قرار گیرد. این سیاست در سیستم های Placement Policy هستند **Best-Fit** و **First-Fit** اهمیت بیشتری دارد، چون قطعات اندازه متغیر دارند. روش های رایج شامل **Segmentation** می شود.

توضیح کامل:

1. تعریف

وقتی یک فرآیند یا بخشی از آن نیاز به قرارگیری در حافظه دارد، سیستم عامل باید تصمیم بگیرد کدام فضای خالی حافظه به آن اختصاص داده شود. این تصمیم توسط Placement Policy گرفته می شود.

2. اهمیت

- در **Paging (صفحه بندی)**: نیاز به Placement Policy خیلی کم است، چون همه ی صفحات و فریم ها اندازه ثابت دارند و هر فریم آزاد می تواند استفاده شود.

- در **Segmentation (قطعه‌بندی)**: هر قطعه اندازه متغیر دارد؛ بنابراین باید دقت کرد که قطعه در فضای مناسب قرار گیرد تا از **Fragmentation** (تکه‌تکه شدن حافظه) جلوگیری شود.

3. روش‌های متداول

- اولین فضای آزاد بزرگ‌تر یا مساوی اندازه قطعه انتخاب می‌شود: **First-Fit (اولین جا)**.
- سریع و ساده، ولی ممکن است فضاهای کوچک بلااستفاده بمانند.
- کوچک‌ترین فضای آزاد که برای قطعه کافی باشد انتخاب می‌شود: **Best-Fit (بهترین جا)**.
- حافظه بهینه‌تر استفاده می‌شود، اما ممکن است سرعت جستجو کند باشد و تکه‌های ریز زیادی تولید کند.
- **Other Policies (سیاست‌های دیگر)**:
 - **Worst-Fit**: بزرگ‌ترین فضای آزاد انتخاب می‌شود (برای کاهش تکه‌های کوچک).
 - **Next-Fit**: جستجو از محل آخرین تخصیص ادامه می‌یابد.

مثال ساده آموزشی:

فرض کن حافظه شامل بلوک‌های خالی زیر باشد: [40 KB] [15 KB] [20 KB] [10 KB]
و یک قطعه 18 KB باید جایگذاری شود:

- **First-Fit**: انتخاب می‌شود (20 KB) اولین فضای کافی.
- **Best-Fit**: انتخاب می‌شود (20 KB) کوچک‌ترین فضای کافی.
- اگر قطعه 12 KB بود:
- **First-Fit**: 20 KB → 8 KB باقی می‌ماند.
- **Best-Fit**: 15 KB → 3 KB باقی می‌ماند.

نمودار (Mermaid) – مقایسه First-Fit و Best-Fit:

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی:

- Placement Policy = تصمیم‌گیری درباره محل قرارگیری قطعه در حافظه.
- اهمیت اصلی در **Segmentation** است (قطعات اندازه متغیر دارند).
- First-Fit → سریع، اما ممکن است فضای کوچک هدر برود.
- Best-Fit → استفاده بهینه‌تر، اما سرعت پایین‌تر و احتمال ایجاد تکه‌های کوچک.
- سیاست‌های دیگر مانند Worst-Fit و Next-Fit نیز وجود دارند.

Replacement Policy (سیاست جایگزینی صفحه)

خلاصه:

مشخص می‌کند **کدام صفحه موجود در حافظه اصلی باید جایگزین شود** زمانی که صفحه جدیدی نیاز است اما حافظه پر Replacement Policy است. این تصمیم شامل سه موضوع کلیدی است: تعداد فریم‌های اختصاصی به هر فرآیند، محدوده انتخاب صفحات برای جایگزینی، و انتخاب صفحه خاص برای حذف local/global.

1. تعریف

وقتی حافظه اصلی پر است و یک فرآیند به صفحه‌ای جدید نیاز دارد (Page Fault)، سیستم‌عامل باید یک صفحه از حافظه را بیرون کند تا جای صفحه جدید باز شود. این سیاست توسط **Replacement Policy** تعیین می‌شود.

2. مسائل کلیدی

- تعداد فریم‌ها برای هر فرآیند (Frame Allocation):
 - آیا هر فرآیند تعداد ثابتی فریم دارد یا می‌تواند متغیر باشد؟
 - تخصیص فریم بیشتر → Page Fault کمتر، اما حافظه کمتری برای بقیه فرآیندها می‌ماند.
- دامنه جایگزینی (Replacement Scope):
 - فقط از صفحات همان فرآیند انتخاب می‌شود: **Local Replacement**.
 - از تمام صفحات موجود در حافظه می‌توان انتخاب کرد (حتی از فرآیندهای دیگر): **Global Replacement**.
- انتخاب صفحه خاص برای حذف:
 - سیاست باید تصمیم بگیرد کدام یک از صفحات کاندید باید جایگزین شود.
 - چون سیستم آینده را نمی‌داند، بیشتر الگوریتم‌ها سعی می‌کنند از رفتار گذشته برای پیش‌بینی آینده استفاده کنند.

3. سیاست‌های رایج

- Optimal**: صفحه‌ای که دیرترین زمان در آینده نیاز خواهد شد. (ایده‌آل، اما غیرعملی).
- FIFO (First-In First-Out)**: قدیمی‌ترین صفحه بارگذاری شده جایگزین می‌شود.
- LRU (Least Recently Used)**: صفحه‌ای که مدت زیادی استفاده نشده جایگزین می‌شود.
- Clock (Second Chance)**: (reference bit) با استفاده از بیت استفاده FIFO نسخه بهبود یافته.

مثال ساده آموزشی:

فرض کن حافظه فقط ۳ فریم دارد و فرآیند دنباله صفحات زیر را تولید می‌کند:

5, 2, 1, 4, 3, 2, 1

- با **FIFO**:
 - ابتدا 1, 2, 3 وارد می‌شوند.
 - وقتی صفحه 4 نیاز شد → صفحه 1 بیرون می‌رود.
 - وقتی دوباره صفحه 1 نیاز شد → Page Fault (چون حذف شده بود).
- با **LRU**:
 - وقتی صفحه 4 نیاز شد → صفحه‌ای که مدت طولانی استفاده نشده (یعنی 1) جایگزین می‌شود.
 - دقت بالاتر نسبت به FIFO.

نمودار (Mermaid) – روند Replacement Policy:

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

- سه پرسش اساسی:
 1. چند فریم به هر فرآیند داده شود؟
 2. جایگزینی محلی باشد یا جهانی؟
 3. کدام صفحه دقیقاً جایگزین شود؟
- بیشتر الگوریتم‌ها از رفتار گذشته برای پیش‌بینی آینده استفاده می‌کنند.
- الگوریتم‌های مهم: Optimal (ایده‌آل)، FIFO (ساده)، LRU (کارآمد)، Clock (بهینه‌سازی‌شده).

قفل کردن قاب‌ها در حافظه – Frame Locking

توضیح کامل و دقیق

توسط سیستم‌عامل یا (main memory frames) به حالتی اشاره دارد که برخی قاب‌های حافظه اصلی (قفل کردن قاب‌ها) **Frame Locking** اجازه ندارد محتوای آن قاب‌ها را تغییر دهد یا (replacement policy) بخش‌های مهم برنامه قفل می‌شوند، به‌طوری‌که الگوریتم جایگزینی جایگزین کند.

این کار به دلایل زیر ضروری است:

1. **محافظت از هسته سیستم‌عامل (Kernel):** بخش‌هایی از هسته سیستم‌عامل باید همواره در حافظه باقی بمانند، زیرا اگر جایگزین شوند، عملکرد سیستم مختل می‌شود.
 2. **ساختارهای کنترلی مهم:** جداول و داده‌های حیاتی مدیریت سیستم (مثل جدول فرآیندها یا جدول صفحه) نباید حذف شوند.
 3. **بافرهای ورودی/خروجی (I/O Buffers):** برخی داده‌ها در حال پردازش لحظه‌ای هستند (مثل اطلاعات در حال انتقال به دیسک یا شبکه) و حذف آن‌ها باعث بروز خطا یا از دست رفتن داده می‌شود.
 4. **بخش‌های حساس به زمان (Time-Critical Areas):** این بخش‌ها نیاز دارند که همیشه در دسترس سریع باشند و نباید با جایگزینی صفحات دچار تأخیر شوند.
- بنابراین، الگوریتم‌های جایگزینی صفحه قبل از تصمیم‌گیری باید بررسی کنند که آیا قاب انتخاب‌شده "قفل" است یا خیر. اگر قفل باشد، آن قاب قابل جایگزینی نیست.

مثال ساده آموزشی

فرض کنید در یک سیستم:

- 5 قاب حافظه وجود دارد.
 - الگوریتم جایگزینی **LRU (Least Recently Used)** استفاده می‌شود.
 - قاب شماره 2 و 4 قفل شده‌اند (چون شامل کد هسته و یک بافر I/O هستند).
- 📌 حال اگر سیستم نیاز به بارگذاری یک صفحه جدید داشته باشد:
- الگوریتم LRU می‌خواهد قاب "قدیمی‌ترین استفاده" را جایگزین کند.
 - اما اگر این قاب در بین قاب‌های **قفل‌شده** باشد (مثلاً قاب شماره 2)، الگوریتم باید آن را نادیده بگیرد و به دنبال قاب دیگری بگردد.

نمودار (Mermaid)

نکات کلیدی

- باعث می‌شود بخش‌های حساس سیستم‌عامل همیشه در حافظه باقی بمانند Frame Locking.
- قفل‌شدن قاب به معنی **محافظت در برابر الگوریتم‌های جایگزینی** است.
- این مکانیزم برای **پایداری، امنیت و سرعت سیستم** ضروری است.
- الگوریتم جایگزینی باید همیشه قاب‌های قفل‌شده را بررسی و رد کند.

الگوریتم‌های پایه جایگزینی صفحه – Basic Algorithms

توضیح کامل و دقیق

📌 صرف‌نظر از این‌که استراتژی مدیریت **Resident Set** (مجموعه صفحاتی که در حافظه اصلی نگهداری می‌شوند) چگونه باشد، سیستم‌عامل برای انتخاب صفحه‌ای که باید جایگزین شود از **الگوریتم‌های جایگزینی صفحه** استفاده می‌کند.

چهار الگوریتم پایه و پرکاربرد عبارت‌اند از:

1. Optimal (بهینه)

- **ایده:** همیشه صفحه‌ای جایگزین می‌شود که در آینده **بیشترین فاصله تا استفاده بعدی** را دارد یا اصلاً دیگر مورد استفاده قرار نخواهد گرفت.
- **مزیت:** کمترین تعداد page fault ممکن.
- **عیب:** در عمل غیرقابل اجراست، چون نیاز به دانستن آینده دارد.
- **کاربرد:** به‌عنوان **مبنای مقایسه** برای سایر الگوریتم‌ها استفاده می‌شود.

2. Least Recently Used (LRU – کم‌ترین استفاده اخیر)

- **ایده:** صفحه‌ای که **طی مدت زمان طولانی‌تری استفاده نشده است**، جایگزین می‌شود.
- **منطق:** گذشته معمولاً نشان‌دهنده آینده است؛ اگر صفحه‌ای مدتی استفاده نشده، احتمال استفاده مجدد آن کم است.
- **پیاده‌سازی:** می‌توان از شمارنده زمانی یا پشته استفاده کرد.
- **مزیت:** عملکرد خوب و نزدیک به Optimal.
- **عیب:** هزینه سخت‌افزاری/نرم‌افزاری بالا برای ردیابی زمان آخرین استفاده.

3. First-In-First-Out (FIFO – صف اول وارد، اول خارج)

- **ایده:** قدیمی‌ترین صفحه (که زودتر از همه وارد حافظه شده است) جایگزین می‌شود.
- **مزیت:** پیاده‌سازی ساده (یک صف کافی است).

- **عیب:** ممکن است صفحاتی که هنوز پرکاربرد هستند، حذف شوند → پدیده **Anomaly of Belady** رخ می‌دهد (افزایش قاب حافظه می‌تواند باعث افزایش Page Fault شود).

4. Clock (ساعت)

- **ایده:** شبیه FIFO است، اما از یک اشاره‌گر دایره‌ای (مثل عقربه ساعت) برای حرکت روی قاب‌ها استفاده می‌کند.
- **پیاده‌سازی:**
 - به هر صفحه یک بیت استفاده (use bit) اختصاص داده می‌شود.
 - هنگام Page Fault، الگوریتم به دنبال صفحه‌ای می‌گردد که بیت استفاده آن 0 باشد.
 - اگر بیت 1 باشد، آن را صفر کرده و عقربه به جلو حرکت می‌کند.
- **مزیت:** کارایی نزدیک به LRU با هزینه کمتر.
- **کاربرد:** پرکاربردترین الگوریتم عملی در سیستم‌عامل‌ها.

مثال ساده آموزشی

فرض کنید حافظه 3 قاب دارد و رشته ارجاعات (Reference String) زیر است:

7, 0, 1, 2, 0, 3, 0, 4

- **FIFO:** صفحه‌های قدیمی ($7 \rightarrow 0 \rightarrow 1$) به ترتیب جایگزین می‌شوند.
- **LRU:** صفحه‌ای که مدت طولانی‌تر استفاده نشده (مثلاً 7) جایگزین می‌شود.
- **Optimal:** صفحه‌ای جایگزین می‌شود که در آینده دیرتر نیاز می‌شود.
- **Clock:** با حرکت عقربه و بیت استفاده تصمیم گرفته می‌شود.

نکات کلیدی

- **Optimal** → بهترین حالت تئوری، اما عملی نیست.
- **LRU** → اما پرهزینه‌تر، Optimal نزدیک به.
- **FIFO** → ساده، اما ممکن است ناکارآمد شود.
- **Clock** → تعادل خوب بین سادگی و کارایی، پرکاربرد در عمل.

سیاست جایگزینی بهینه – Optimal Policy

توضیح کامل و دقیق

📌 **سیاست بهینه (Optimal Replacement Policy)** یکی از الگوریتم‌های مهم در مدیریت حافظه است که به‌عنوان یک **استاندارد تئوری** برای مقایسه سایر الگوریتم‌ها مطرح می‌شود.

- **ایده اصلی:** هنگام وقوع **Page Fault**، باید آن صفحه‌ای از حافظه اصلی حذف شود که **بیشترین فاصله زمانی تا استفاده بعدی** را دارد. به بیان ساده: اگر بدانیم کدام صفحه در آینده دیرتر از همه مورد نیاز قرار می‌گیرد، آن صفحه بهترین انتخاب برای حذف است.

- مزیت: تعداد Page Fault ها را به حداقل ممکن می‌رساند.
- عیب بزرگ: در عمل غیرقابل پیاده‌سازی است، چون سیستم عامل نمی‌تواند از آینده خبر داشته باشد.

مثال ساده آموزشی

فرض کنید حافظه 3 قاب دارد و رشته ارجاعات این است:

4 , 0 , 3 , 0 , 2 , 1 , 0 , 7

در لحظه‌ای که نیاز به جایگزینی داریم (مثلاً بعد از پر شدن حافظه)، الگوریتم Optimal به این شکل عمل می‌کند:

- اگر قاب‌ها پر شده باشند (مثلاً شامل [1, 0, 7]) و عدد بعدی در رشته 2 باشد → باید یکی حذف شود.
- الگوریتم Optimal نگاه می‌کند که در آینده:
 - 7 دیگر تکرار نمی‌شود،
 - 0 به زودی تکرار می‌شود،
 - 1 بعداً دوباره می‌آید.
- بنابراین صفحه 7 را حذف می‌کند، چون دیرترین یا هیچ‌وقت استفاده نمی‌شود.

نمودار

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- الگوریتمی تئوری و غیرقابل اجرا در عمل → Optimal.
- معیار انتخاب: بیشترین فاصله تا استفاده بعدی.
- بهترین معیار برای مقایسه سایر الگوریتم‌ها (مثل FIFO و LRU).
- همیشه کمترین تعداد Page Fault را تضمین می‌کند.

Least Recently Used (LRU) Page Replacement Policy

توضیح:

- هدف: جایگزینی صفحه‌ای که طولانی‌ترین زمان از آخرین استفاده آن گذشته است.
- اصل کار: بر اساس Principle of Locality (اصل مکان‌یابی محلی): صفحاتی که اخیراً استفاده نشده‌اند، احتمالاً در آینده نزدیک کمتر مورد استفاده قرار می‌گیرند.
- عملکرد: تقریباً به اندازه الگوریتم Optimal عمل می‌کند و تعداد Page Fault کمی ایجاد می‌کند.

مشکلات و محدودیت‌ها:

1. پیچیدگی پیاده‌سازی:

- هر صفحه باید با **زمان آخرین ارجاع** برچسب‌گذاری شود.
- این برچسب‌گذاری باید در هر دسترسی به حافظه (دستورات و داده‌ها) انجام شود.
- حتی اگر سخت‌افزار از این کار پشتیبانی کند، **بار پردازشی بسیار زیاد** خواهد بود.

2. راه‌حل جایگزین:

- نگهداری یک **stack از ارجاعات صفحات**، که همیشه صفحات اخیراً استفاده شده در بالای stack باشند.
- هنگام نیاز به جایگزینی، صفحه پایین‌ترین سطح stack (کمترین استفاده اخیر) انتخاب می‌شود.

مثال ساده آموزشی

فرض کنید ترتیب دسترسی به صفحات:

5 , 1 , 4 , 2 , 3 , 2 , 1

و حافظه ظرفیت 3 صفحه دارد.

- وقتی نیاز به جایگزینی است، **صفحه‌ای که کمترین استفاده اخیر را داشته انتخاب می‌شود**.
- ایجاد می‌کند، چون از اطلاعات گذشته استفاده می‌کند **FIFO کمتر از Page Fault** معمولاً LRU.

نمودار Mermaid – نحوه کار LRU

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- جایگزینی کم‌استفاده‌ترین صفحه اخیر: **LRU**.
- **اصل:** Principle of Locality (صفحات اخیراً استفاده نشده احتمالاً کمتر استفاده می‌شوند).
- **عملکرد:** نزدیک به Optimal.
- **پیچیدگی پیاده‌سازی:** نیاز به ثبت زمان آخرین دسترسی یا نگهداری stack، بار پردازشی زیاد.

First-In-First-Out (FIFO) Page Replacement Policy

توضیح:

- **روش کار:**
 - فریم‌های صفحه اختصاص داده شده به یک فرآیند به صورت **حلقوی (Circular Buffer)** در نظر گرفته می‌شوند.
 - صفحات **به ترتیب ورود** و به سبک **Round-Robin** جایگزین می‌شوند.
- **مزیت اصلی:**
 - یکی از **ساده‌ترین الگوریتم‌ها برای پیاده‌سازی** است.
- **منطق انتخاب:**
 - صفحه‌ای که **طولانی‌ترین زمان در حافظه بوده** جایگزین می‌شود.
 - فرض بر این است که صفحه‌ای که مدت زیادی در حافظه بوده، ممکن است دیگر استفاده نشود.

مثال ساده آموزشی

فرض کنید حافظه ظرفیت 3 صفحه دارد و دسترسی به صفحات به ترتیب زیر است:

5 , 2 , 1 , 4 , 3 , 2 , 1

- وقتی صفحه جدید وارد می‌شود و حافظه پر است:
- اولین صفحه‌ای که وارد حافظه شده است، جایگزین می‌شود: **FIFO**.
- در این مثال:
 - هنگام ورود صفحه 4، صفحه 1 جایگزین می‌شود.
 - هنگام ورود صفحه 5، صفحه 2 جایگزین می‌شود.

⚠️ مشکل مهم FIFO: ممکن است صفحه‌ای که هنوز زیاد استفاده می‌شود، زود جایگزین شود (*Belady's anomaly*).

نمودار Mermaid – نحوه کار FIFO

!Error parsing Mermaid diagram

Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- جایگزینی قدیمی‌ترین صفحه وارد شده در حافظه: **FIFO**.
- مزیت:** ساده و راحت برای پیاده‌سازی.
- معایب:** ممکن است صفحه‌ای که هنوز استفاده می‌شود، زود جایگزین شود (*Belady's anomaly*).
- مکانیزم:** استفاده از Round-Robin و Circular Buffer.

Clock Page Replacement Policy

توضیح:

- است LRU یکی از پیاده‌سازی‌های عملی و کم‌هزینه Clock.
- مکانیزم اصلی:**
 - به هر فریم حافظه یک **Use Bit** (یا **Referenced Bit**) اختصاص داده می‌شود.
 - وقتی صفحه‌ای وارد فریم می‌شود، **Use Bit** آن 1 می‌شود.
 - هر بار که صفحه ارجاع داده شود (پس از Page Fault اولیه)، **Use Bit** دوباره 1 می‌شود.
- انتخاب صفحه برای جایگزینی:**
 - فریم‌ها به صورت **دایره‌ای (Circular)** مرور می‌شوند.
 - هنگام نیاز به جایگزینی، سیستم به دنبال فریمی با **Use Bit = 0** می‌گردد.
 - اگر **Use Bit = 1** باشد، آن را به 0 تغییر می‌دهد و ادامه می‌دهد تا فریم مناسب پیدا شود.
- مزیت:**
 - نزدیک به LRU عمل می‌کند ولی پیاده‌سازی ساده‌تر و کم‌هزینه‌تر دارد.

مثال ساده آموزشی

فرض کنید حافظه ظرفیت 4 صفحه دارد و دسترسی به صفحات به ترتیب زیر است:

5 , 4 , 1 , 3 , 2 , 1

- همه فریم‌ها به صورت دایره‌ای مرتب شده‌اند و **Use Bit** هر فریم مدیریت می‌شود.
- هنگام ورود صفحه جدید:
 1. سیستم از Pointer شروع به بررسی فریم‌ها می‌کند.
 2. اگر $\text{Use Bit} = 0$ باشد، آن فریم جایگزین می‌شود.
 3. اگر $\text{Use Bit} = 1$ باشد، آن را به 0 تغییر می‌دهد و به فریم بعدی می‌رود.

نمودار Mermaid – نحوه کار Clock

!Error parsing Mermaid diagram

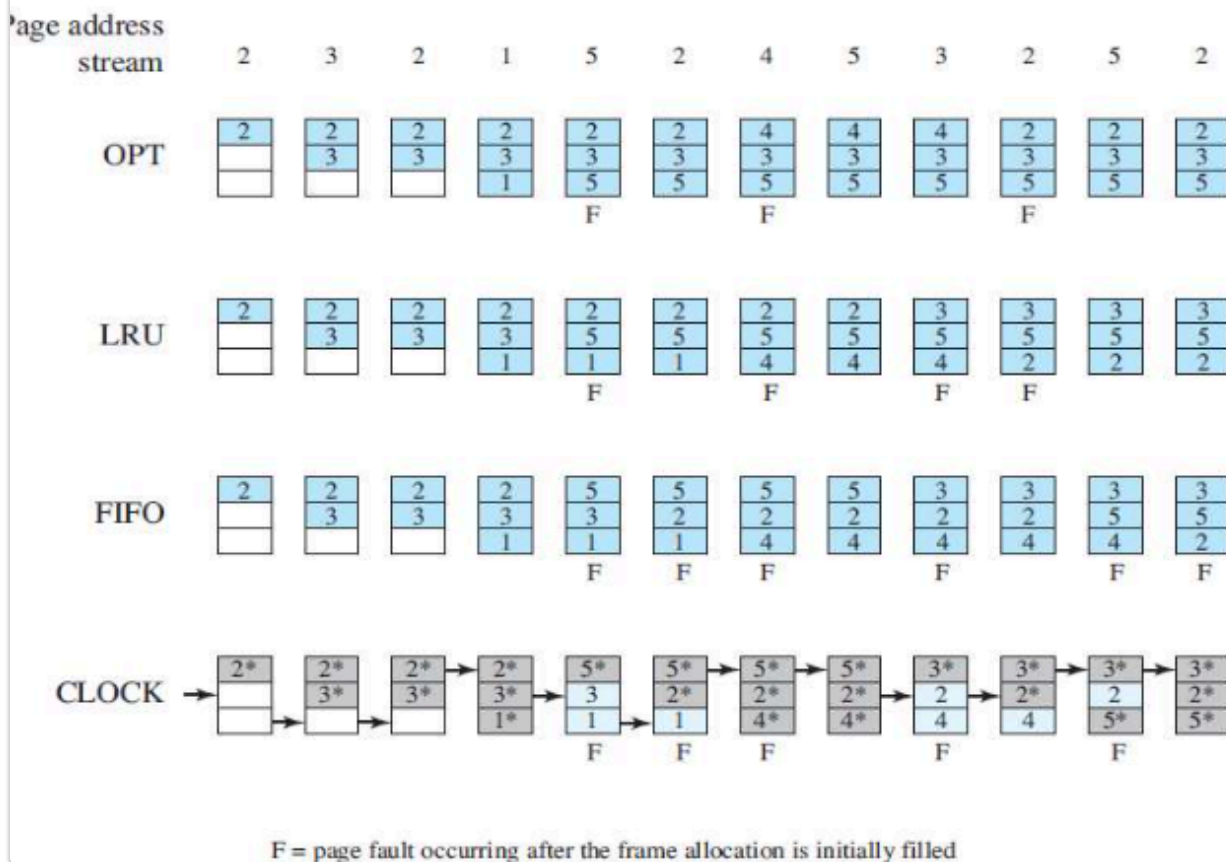
Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- **Clock Policy:** نسخه عملی LRU با Use Bit.
- **Use Bit:** نشان‌دهنده ارجاع اخیر صفحه.
- **مکانیزم:** اسکن دایره‌ای فریم‌ها و جایگزینی اولین فریم با $\text{Use Bit} = 0$.
- **مزیت:** کم‌هزینه و ساده نسبت به پیاده‌سازی کامل LRU.
- **کاربرد عملی:** در سیستم‌عامل‌های واقعی، الگوریتم Clock بسیار محبوب است.



Behavior of Four Page Replacement Algorithms



Behavior of Four Page Replacement Algorithms

توضیح تصویر:

- **Page address stream:** (2, 5, 2, 3, 5, 4, 2, 5, 1, 2, 3, 2) رشته دسترسی به صفحات
- **حافظه:** ظرفیت 3 فریم دارد.
- **F (Fault):** (وقتی صفحه در حافظه موجود نیست و باید بارگذاری شود) **Page Fault** نشان‌دهنده رخ دادن

1. Optimal (OPT)

- صفحه‌ای که **دیرترین استفاده در آینده** دارد جایگزین می‌شود.
- نتیجه: **حداقل تعداد Page Fault** (در تصویر 5 بار).
- کارایی ایده‌آل اما در عمل غیرقابل پیاده‌سازی.

2. Least Recently Used (LRU)

- صفحه‌ای که **طولانی‌ترین زمان از آخرین استفاده‌اش گذشته** جایگزین می‌شود.
- عملکرد نزدیک به OPT.
- نتیجه: **Page Fault 6**.
- نسبت به FIFO بهتر است چون از اصل Locality استفاده می‌کند.

3. First-In-First-Out (FIFO)

- قدیمی‌ترین صفحه وارد شده جایگزین می‌شود.
- نتیجه: **Page Fault 7**.
- مشکل: ممکن است صفحه‌ای که هنوز لازم است، حذف شود (**Belady's anomaly**).

4. Clock

- شبیه LRU ولی ساده‌تر:
- از **Use Bit** و **Pointer** دایره‌ای استفاده می‌کند.
- نتیجه: **Page Fault 6** (در این مثال برابر با LRU).
- در عمل کاربردی‌تر از LRU به خاطر هزینه کمتر.

مقایسه تعداد Page Fault

تعداد Page Fault	الگوریتم
5 (کمترین)	OPT
6	LRU
6	Clock
7 (بیشترین)	FIFO

نمودار Mermaid – مقایسه الگوریتم‌ها

!Error parsing Mermaid diagram

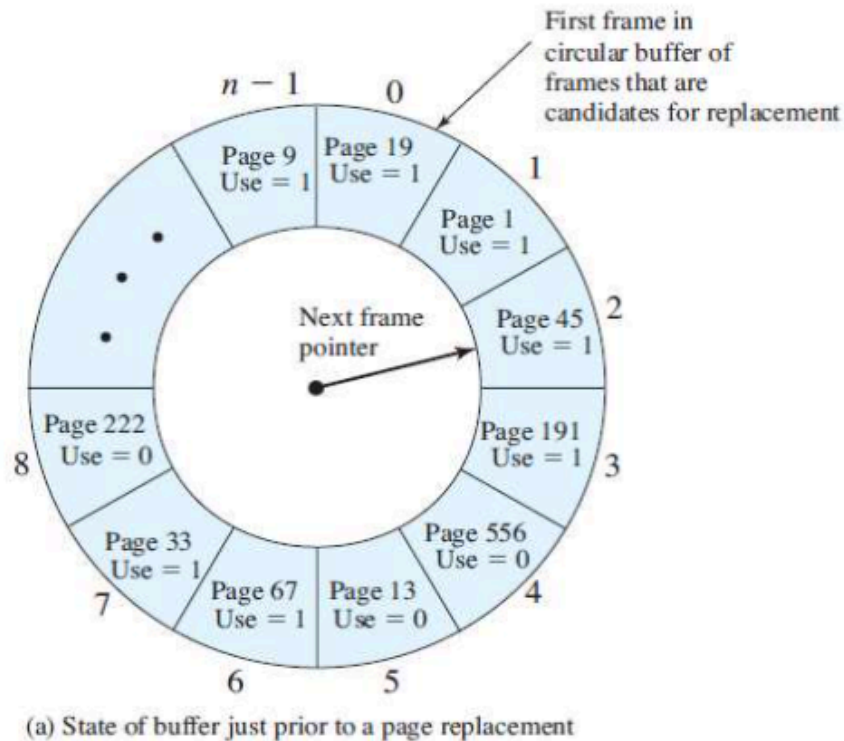
Cannot read properties of null (reading 'getBoundingClientRect')

نکات کلیدی

- **OPT**: کمترین خطا، اما غیرقابل پیاده‌سازی واقعی.
- **LRU**: ولی پرهزینه OPT بهترین الگوریتم عملی، نزدیک به.
- **FIFO**: ساده‌ترین، اما بیشترین Page Fault.
- **Clock**: در سیستم‌های واقعی LRU الگوریتمی عملی و کارآمد، جایگزین خوب.



Clock



توضیح شکل

در این شکل یک بافر دایره‌ای از فریم‌های حافظه داریم (شبیه عقربه ساعت):

- هر خانه یک Frame است که داخلش یک Page نگهداری می‌شود.
- هر صفحه علاوه بر شماره‌اش، یک Use Bit (بیت استفاده) هم دارد (0 یا 1).
- مثل یک عقربه است که همیشه نشان می‌دهد نوبت بررسی کدام فریم رسیده **Next Frame Pointer**.

نحوه کار الگوریتم:

1. وقتی یک Page Fault رخ بدهد، OS باید تصمیم بگیرد کدام صفحه را بیرون بیندازد.
2. عقربه (Next Frame Pointer) شروع به حرکت در حلقه می‌کند:
 - اگر **Use Bit = 1** باشد → سیستم آن را به 0 تغییر می‌دهد و به فریم بعدی می‌رود.
 - اگر **Use Bit = 0** باشد → آن صفحه انتخاب می‌شود و جایگزینی انجام می‌شود.
3. پس از جایگزینی، عقربه یک خانه جلو می‌رود و همان‌جا می‌ماند تا برای دفعه بعد شروع کند.

وضعیت تصویر:

- در فریم‌ها صفحاتی مثل Page 9، Page 19، Page 1، Page 45 و غیره قرار دارند.
- بیشتر Use Bit ها برابر 1 هستند (یعنی اخیراً استفاده شده‌اند).
- تنها **Page 222 (Use = 0)** آماده جایگزینی است.
- را پیدا کند **Use Bit = 0** حرکت می‌کند تا اولین pointer، **ایستاده** و اگر لازم باشد صفحه‌ای جایگزین شود **Page 45 روی Pointer**.
- پس از گذر از چند فریم، به **Page 222** می‌رسد (**Use = 0**) → این صفحه انتخاب و جایگزین می‌شود.

مزایا و معایب الگوریتم ساعت

مزایا:

- ساده‌تر و کم‌هزینه‌تر از LRU.
- عملکرد نزدیک به LRU در عمل.
- نیاز به نگهداری دقیق زمان یا صف ندارد، فقط یک بیت ساده کافی است.

معایب:

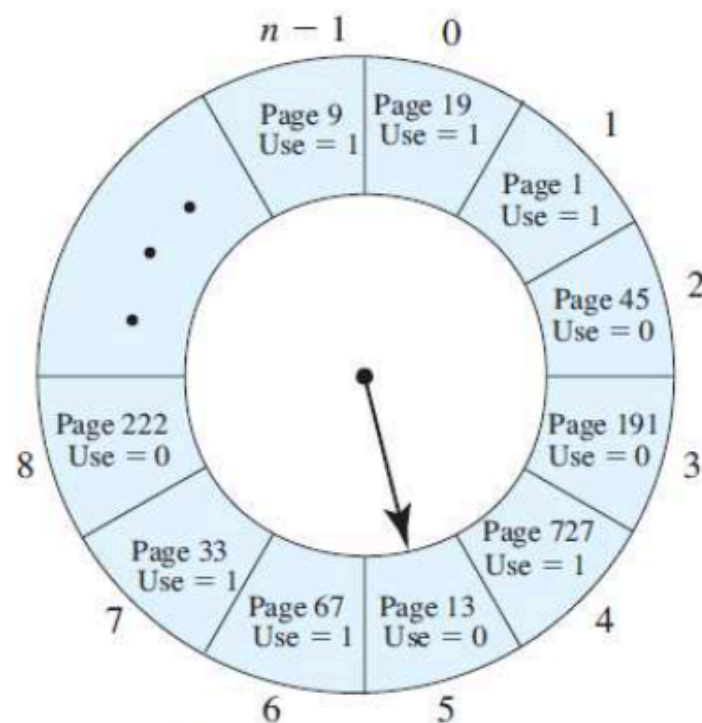
- همیشه به خوبی LRU عمل نمی‌کند.
- اگر بیشتر بیت‌ها = 1 باشند، pointer باید کل حلقه را دور بزند تا یک 0 پیدا کند (کندتر می‌شود).

نکات کلیدی

- الگوریتم ساعت نسخه‌ای ساده و کارآمد از LRU است.
- از **Use Bit** و **Pointer دایره‌ای** استفاده می‌کند.
- در هر جایگزینی → صفحه با $\text{Use Bit} = 0$ جایگزین می‌شود.
- عملکرد بین FIFO و LRU قرار دارد.



Clock



(b) State of buffer just after the next page replacement



توضیح شکل – حالت «just after the next page replacement»

- تصویر یک حلقه از فریم‌ها (مثل صفحه‌های ساعت) نشان می‌دهد که هر فریم شامل:
- شماره صفحه (مثلاً Page 19, Page 1, Page 45, ...)

- آن (0 یا 1) **Use bit**

- مکان شروع اسکن برای جایگزینی بعدی را نشان می‌دهد (عقربه) **Pointer**.
- عنوان تصویر می‌گوید این تصویر بلافاصله بعد از انجام یک جایگزینی صفحه است – یعنی الگوریتم Clock تازه یک صفحه را جایگزین کرده و وضعیت فریم‌ها/بیت‌ها را به‌روزرسانی کرده است.

آنچه معمولاً دقیقاً اتفاق افتاده (روند استاندارد Clock که منطبق با این شکل است):

1. هنگام Page Fault، سیستم از مکان Pointer شروع به اسکن دایره‌ای می‌کند.
2. برای هر فریمی که $Use\ bit = 1$ است:
 - آن را **ریست (به 0)** می‌کند و Pointer را به فریم بعدی می‌برد.
3. تا زمانی که فریمی با $Use\ bit = 0$ پیدا شود.
4. وقتی فریم با $Use\ bit = 0$ یافت شد:
 - آن فریم **جایگزین** می‌شود (صفحه جدید در آن قرار می‌گیرد).
- صفحه‌ی تازه‌وارد معمولاً 1 قرار داده می‌شود (زیرا همین‌الآن بارگذاری و احتمالاً استفاده خواهد شد) $Use\ bit$.
- پس از جایگزینی معمولاً به فریم **بعدی** منتقل می‌شود – و آن نقطه، نقطه شروع اسکن بعدی خواهد بود Pointer.
5. **حالت «just after replacement»** نشان می‌دهد که:
 - یک فریم با $Use=0$ انتخاب و جایگزین شده،
 - برخی فریم‌هایی که هنگام اسکن قبلی $Use=1$ داشته‌اند به 0 ریست شده‌اند،
 - الان به اولین فریم بعد از آن جایگزینی اشاره می‌کند Pointer.

تفسیر منطقی تصویر (چند نکته قابل نوشتن در جزوه)

- اگر در تصویر قبل از این یک یا چند $Use=1$ وجود داشتند، الگوریتم هنگام اسکن آن‌ها را به 0 ریست کرد تا بالاخره به یک $Use=0$ رسید – و آن $Use=0$ همان صفحه‌ای بود که حذف و با صفحه جدید جایگزین شد.
- بعد از جایگزینی:
 - صفحه جدید معمولاً $Use=1$ می‌گیرد (معمولاً در بسیاری پیاده‌سازی‌ها هنگام بارگذاری اولیه 1 می‌شود).
 - به فریم بعدی (برای اسکن بعدی) حرکت کرده است – همان نقطه‌ای که در تصویر دیده می‌شود Pointer.

مثال گام‌به‌گام کوچک

فرض کنید قبل از جایگزینی وضعیت (از موقعیت Pointer) به‌صورت زیر بوده:

... ← Pointer → [Page 45, Use=1] → [Page 191, Use=1] → [Page 556, Use=0]

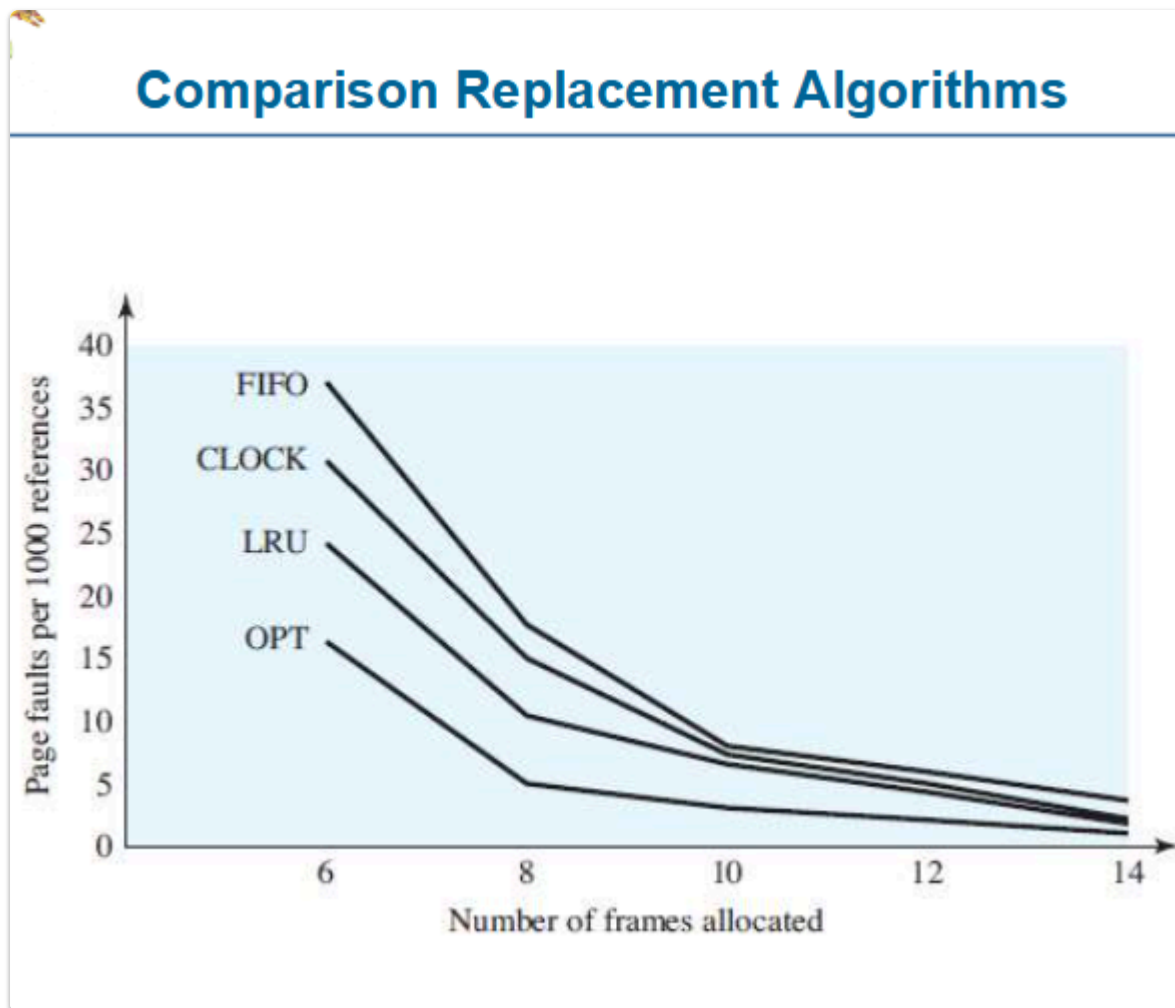
الگوریتم اسکن می‌کند:

- می‌بیند Page45 Use=1 → آن را به 0 می‌کند و می‌رود جلو.
- Page191 Use=1 → آن را به 0 می‌کند و می‌رود جلو.
- Page556 Use=0 → این فریم انتخاب می‌شود و جایگزین می‌گردد → (مثلاً با Page X).
- پس از جایگزینی:
 - $Use(X) = 1$ ، وارد می‌شود Page X
 - می‌ایستد (برای دفعه بعد) Page556 روی فریم بعد از Pointer.

این همان چیزی است که نمودار «just after replacement» نشان می‌دهد: بعضی Use‌ها ریست شده‌اند، یک فریم جایگزین شده و pointer جلوتر رفته است.

نکات کلیدی

- الگوریتم **Clock** با یک **Use bit** به‌ازای هر فریم کار می‌کند و اسکن دایره‌ای انجام می‌دهد.
- هنگام اسکن، اگر $Use=1$ باشد آن را به 0 ریست می‌کند؛ اگر $Use=0$ باشد آن فریم را جایگزین می‌کند.
- بعد از جایگزینی، صفحه جدید معمولاً $Use=1$ می‌گیرد و pointer به فریم بعدی منتقل می‌شود.
- تصویر «just after replacement» نشان‌دهنده این است که الگوریتم تازه یک فریم با $Use=0$ را یافته و جایگزین کرده و برخی Use ها را نیز صفر کرده است.
- است؛ نگهداری فقط 1 بیت برای هر فریم کافی است LRU یک پیاده‌سازی کم‌هزینه نزدیک به Clock.



این شکل مقایسه‌ای بین الگوریتم‌های جایگزینی صفحه (Page Replacement Algorithms) نشان می‌دهد.

توضیح محورهای شکل

- **محور افقی (X-axis):** تعداد قاب‌های (Frames) اختصاص داده شده به یک فرآیند.
- **محور عمودی (Y-axis):** تعداد **Page Faults** (خطاهای صفحه) به ازای هر 1000 دسترسی.

نکات کلیدی در شکل

1. FIFO (First-In-First-Out):

- بالاترین نرخ Page Fault رو دارد.
- چون صرفاً قدیمی‌ترین صفحه حذف می‌شود، حتی اگر هنوز پرکاربرد باشد.
- خط روی نمودار مربوط به FIFO بالاتر از همه است.

2. CLOCK Algorithm:

- بهبود یافته‌ی FIFO هست.
- از یک بیت مرجع استفاده می‌کنه تا بفهمه آیا صفحه اخیراً استفاده شده یا نه.
- عملکردش بهتر از FIFO ولی هنوز ضعیف‌تر از LRU و OPT هست.

3. LRU (Least Recently Used):

- صفحه‌ای که **قدیمی‌ترین استفاده** رو داشته حذف می‌کنه.
- عملکردش نزدیک به حالت بهینه (OPT) هست.
- نرخ Page Fault کمتر از FIFO و CLOCK.

4. OPT (Optimal):

- الگوریتم ایده‌آل (تئوری).
- همیشه صفحه‌ای رو حذف می‌کنه که در آینده دورترین زمان ممکن دوباره نیاز میشه.
- در عمل پیاده‌سازی نمی‌شه چون نیاز به **دانستن آینده** داره، ولی به عنوان معیار مقایسه استفاده میشه.
- کمترین Page Fault را دارد (پایین‌ترین خط روی نمودار).

نتیجه‌گیری

- هرچه تعداد قاب‌های حافظه بیشتر باشه، **Page Fault** کاهش پیدا می‌کنه.
- **OPT → LRU → CLOCK → FIFO** بهترین عملکرد OPT.
- ساده است ولی کارایی ضعیف‌تری داره FIFO.
- ارائه می‌ده (OPT) نزدیک‌ترین عملکرد رو به الگوریتم بهینه LRU.

خیلی خوب، توضیح می‌دم 📌

Resident Set Size (اندازه مجموعه مقیم)

زمانی که از **Virtual Memory** و **Paging** استفاده می‌کنیم، هر پروسه برای اجرا نیاز به تعدادی **Frame** در حافظه اصلی داره. تعداد فریم‌هایی که به یک پروسه اختصاص داده می‌شه رو **Resident Set Size** می‌گیم.

1. Fixed-allocation policy (سیاست تخصیص ثابت)

- به یک فرآیند تعداد مشخصی فریم داده میشه (مثلاً 4 فریم).
- این تعداد فریم در طول اجرای فرآیند **تغییر نمی‌کنه**.
- اگه Page Fault رخ بده، یکی از صفحات فعلی پروسه باید جایگزین صفحه جدید بشه.
- **مزیت:** مدیریت ساده و سرشار کمتر برای سیستم‌عامل.
- **عیب:** ممکنه فرآیند در بعضی لحظات نیاز به فریم بیشتری داشته باشه و نتونه کارآمد عمل کنه → افزایش Page Fault.

2. Variable-allocation policy (سیاست تخصیص متغیر)

- تعداد فریم‌های اختصاص داده شده به یک فرآیند می‌تونه در طول زمان تغییر کنه.
- سیستم‌عامل می‌تونه بر اساس **نیاز پروسه** یا **شرایط کلی حافظه** تعداد فریم‌ها رو افزایش یا کاهش بده.
- **مزیت:** استفاده بهتر از حافظه و کاهش Page Fault در مواقع نیاز.
- **عیب:** سرشار نرم‌افزاری بیشتر برای سیستم‌عامل (چون باید مدام تصمیم‌گیری و مدیریت انجام بده).

- **Fixed Allocation** → ساده‌تر ولی انعطاف‌ناپذیر.
- **Variable Allocation** → کارآمدتر ولی نیازمند مدیریت پیچیده‌تر توسط سیستم‌عامل.

🔄 Replacement Scope

1. Local Replacement Policy (سیاست جایگزینی محلی)

- فقط بین **صفحات همان پروسه‌ای** که Page Fault ایجاد کرده انتخاب می‌کند.
- پس: هر پروسه فقط درون فریم‌های خودش جابه‌جایی انجام می‌دهد.
- 📌 **مزیت:** پروسه‌ها روی همدیگه اثر نمی‌ذارن.
- 📌 **عیب:** اگر پروسه نیاز به فریم بیشتری داشته باشه، نمی‌تونه از حافظه استفاده بهینه بکنه.

2. Global Replacement Policy (سیاست جایگزینی سراسری)

- می‌تونه **هر صفحه‌ای در حافظه اصلی** رو جایگزین کنه، حتی اگر متعلق به پروسه‌های دیگه باشه.
- صفحه‌ای که جایگزین میشه، ممکنه از یک پروسه متفاوت باشه.
- 📌 **مزیت:** استفاده بهینه‌تر از حافظه.
- 📌 **عیب:** ممکنه باعث نوسان عملکرد پروسه‌ها بشه (یک پروسه باعث افزایش Page Fault پروسه دیگه بشه).

📊 جدول مقایسه (طبق تصویر)

Global Replacement	Local Replacement	سیاست تخصیص
غیرممکن	تعداد فریم‌ها برای پروسه ثابت هست → فقط از بین همون فریم‌ها انتخاب میشه	Fixed Allocation
از کل فریم‌های موجود حافظه انتخاب میشه → اندازه Resident Set پروسه‌ها تغییر می‌کنه	تعداد فریم‌ها می‌تونه تغییر کنه ولی باز هم جایگزینی فقط درون فریم‌های همون پروسه انجام میشه	Variable Allocation

- **Local** → محدود، کنترل‌شده، پروسه‌ها مستقل.
- **Global** → آزاد، کارایی بیشتر، ولی ممکنه باعث تداخل بین پروسه‌ها بشه.

اسلاید ارائه شده درباره **استراتژی مجموعه کاری (Working Set Strategy)** صحبت می‌کند که یکی از روش‌های مدیریت حافظه در سیستم‌های عامل به حساب می‌آید. این استراتژی بخشی از تکنیک‌های تخصیص متغیر و محلی (Variable-Allocation, Local-Scope Strategy) هست که برای تصمیم‌گیری درباره اندازه مجموعه ساکن (Resident Set) و زمان تغییرش استفاده می‌شود. مجموعه کاری (Working Set) در واقع مجموعه‌ای از صفحات حافظه‌ست که به فرآیند در به بازه زمانی مشخص بهشون نیاز دارد.

در این اسلاید، چند نکته کلیدی مطرح شده:

- مجموعه کاری $(W(t, \Delta))$:** این مجموعه شامل صفحاتی از به فرآیند هست که در آخرین واحدهای زمان مجازی (Virtual Time Units) بهشون مراجعه شده. پارامتر Δ اندازه پنجره زمانی رو مشخص می‌کند که نشون می‌ده چقدر به عقب نگاه کنیم.
- تأثیر اندازه پنجره (Δ) :** هر چی پنجره بزرگ‌تر باشه، مجموعه کاری هم بزرگ‌تر می‌شه. این یعنی اگر Δ رو یک واحد افزایش بدیم $(\Delta + 1)$ ، مجموعه کاری جدید شامل مجموعه کاری قبلی به اضافه صفحات جدید می‌شه $(W(t, \Delta + 1) \supseteq W(t, \Delta))$.
- رشد مجموعه کاری:** اگه به فرآیند به صفحات زیادی به سرعت دسترسی پیدا کنه، مجموعه کاری می‌تونه تا تعداد کل صفحات فرآیند (N) رشد کنه، به شرطی که اندازه پنجره (Δ) به اندازه کافی بزرگ باشه.
- محدوده مجموعه کاری:** اندازه مجموعه کاری همیشه بین 1 (حداقل) و حداقل از Δ یا N (تعداد کل صفحات) هست.

این مفاهیم در فصل مدیریت حافظه (Memory Management) کتاب Silberschatz, Galvin and Gagne، به خصوص در بخش‌هایی که به الگوریتم‌های جایگزینی صفحه (Page Replacement Algorithms) و استراتژی‌های محلی پرداخته شده، توضیح داده شده.

این دیاگرام نشون می‌ده که با افزایش پنجره زمانی، مجموعه کاری گسترش پیدا می‌کند و تابع محدودیت‌های مشخصی (بین 1 و $\min(\Delta, N)$) هست.

مثال ساده آموزشی

فرض کن به فرآیند داره با 5 صفحه حافظه $(N = 5)$ کار می‌کنه. اگه پنجره زمانی (Δ) رو 3 واحد تنظیم کنیم:

- در زمان t ، فرض کنیم فرآیند فقط به صفحات 1، 2 و 3 مراجعه کرده. پس $W(t, 3) = \{1, 2, 3\}$.
- حالا اگه Δ رو به 4 افزایش بدیم، سیستم به 4 واحد زمان گذشته نگاه می‌کنه. اگه صفحه 4 هم استفاده شده باشه، $W(t, 4) = \{1, 2, 3, 4\}$ می‌شه که بزرگ‌تر از قبل هست.
- اگه فرآیند به سرعت به همه 5 صفحه دسترسی پیدا کنه و Δ کافی بزرگ باشه (مثلاً 5)، مجموعه کاری می‌تونه به $\{1, 2, 3, 4, 5\}$ برسه.

نکات کلیدی

- مجموعه کاری (Working Set) شامل صفحاتی هست که فرآیند در به بازه زمانی مشخص بهشون نیاز داره.
- اندازه پنجره (Δ) روی تعداد صفحات مجموعه کاری تأثیر مستقیم داره.
- مجموعه کاری می‌تونه از 1 صفحه شروع بشه و تا حداقل Δ یا تعداد کل صفحات (N) رشد کنه.
- افزایش Δ باعث بزرگ‌تر شدن مجموعه کاری می‌شه، به شرطی که صفحات جدید استفاده بشن.



Working set strategy

Sequence of Page References	Window Size, Δ			
	2	3	4	5
24	24	24	24	24
15	24 15	24 15	24 15	24 15
18	15 18	24 15 18	24 15 18	24 15 18
23	18 23	15 18 23	24 15 18 23	24 15 18 23
24	23 24	18 23 24	•	•
17	24 17	23 24 17	18 23 24 17	15 18 23 24 17
18	17 18	24 17 18	•	18 23 24 17
24	18 24	•	24 17 18	•
18	•	18 24	•	24 17 18
17	18 17	24 18 17	•	•
17	17	18 17	•	•
15	17 15	17 15	18 17 15	24 18 17 15
24	15 24	17 15 24	17 15 24	•
17	24 17	•	•	17 15 24
24	•	24 17	•	•
18	24 18	17 24 18	17 24 18	15 17 24 18

توضیح کامل و قابل فهم از محتوای اسلاید

این اسلاید به ادامه مبحث **استراتژی مجموعه کاری (Working Set Strategy)** در مدیریت حافظه سیستم عامل می پردازد و به مثال عملی نشون می ده. در اینجا، به دنباله از ارجاعات به صفحات حافظه (Page References) داده شده و برای اندازه های مختلف پنجره زمانی (Window Size)، یعنی 2، 3 و 5، مجموعه کاری (Working Set) محاسبه شده. هدف این استراتژی اینه که ببینیم با توجه به الگوی دسترسی به صفحات، کدوم صفحات توی یه بازه زمانی مشخص استفاده می شن تا مدیریت بهتری روی حافظه انجام بشه.

- **دنباله ارجاعات:** لیست ارجاعات به صفحات از 24 شروع می شه و با ترتیب خاصی تا 18 ادامه پیدا می کنه (مثل 15، 18، 23 و ...).
- **اندازه پنجره (Δ):** این عدد نشون می ده که سیستم به چند ارجاع قبلی نگاه می کنه تا مجموعه کاری رو مشخص کنه. مثلاً اگه $\Delta = 2$ باشه، فقط دو ارجاع آخر رو در نظر می گیریم.
- **مجموعه کاری:** ستون ها نشون می دن که برای هر لحظه، مجموعه کاری چیه. مثلاً برای $\Delta = 2$ ، وقتی ارجاع به 24 و بعد 15 می رسه، مجموعه کاری {15، 24} می شه.

با بزرگ تر شدن Δ (مثلاً از 2 به 5)، مجموعه کاری می تونه شامل صفحات بیشتری بشه چون بازه زمانی بیشتری بررسی می شه. این با مفهوم فصل مدیریت حافظه در کتاب Silberschatz, Galvin and Gagne، که به الگوریتم های جایگزینی صفحه و استراتژی های محلی پرداخته، هم راستاست.

مثال ساده آموزشی

فرض کن یه فرآیند دنباله ارجاعات 24، 15، 18 رو داره:

- اگه $\Delta = 2$ باشه:
 - بعد از ارجاع اول (24): $W = \{24\}$
 - بعد از ارجاع دوم (15): $W = \{24, 15\}$
 - بعد از ارجاع سوم (18): $W = \{15, 18\}$ (چون فقط دو ارجاع آخر رو می بینیم).
 - اگه $\Delta = 3$ باشه:
 - بعد از ارجاع سوم (18): $W = \{24, 15, 18\}$ (سه ارجاع آخر رو شامل می شه).
- این نشون می ده که با بزرگ تر شدن Δ ، سیستم حافظه بیشتری رو برای فرآیند رزرو می کنه.

- مجموعه کاری (Working Set) بر اساس اندازه پنجره (Δ) و ارجاعات اخیر به صفحات تعیین می‌شود.
- با افزایش Δ ، مجموعه کاری می‌تونه شامل صفحات بیشتری بشه.
- این استراتژی به سیستم کمک می‌کنه تا صفحاتی که واقعاً استفاده می‌شن رو توی حافظه نگه داره.
- اگه Δ خیلی کوچیک باشه، ممکنه صفحات مهم از مجموعه کاری حذف بشن.

خیلی خوب ✨ این متن توضیح **استراتژی مجموعه کاری (Working Set Strategy)** رو تکمیل می‌کنه.

◆ استراتژی مجموعه کاری (Working Set Strategy)

✓ ایده اصلی

- هدف اینه که **اندازه‌ی resident set** (تعداد فریم‌های داده‌شده به یک پروسه) رو بر اساس **مجموعه کاری واقعی پروسه** تنظیم کنیم.
- مجموعه کاری یعنی همون صفحاتی که پروسه اخیراً استفاده کرده و احتمال داره دوباره استفاده کنه.

📌 روش کار

1. مانیتور کردن Working Set هر پروسه
سیستم عامل پیگیری می‌کنه چه صفحاتی در بازه‌ی Δ اخیراً استفاده شدن.
2. حذف صفحات غیر ضروری
هر چند وقت یکبار، صفحاتی که دیگه توی Working Set نیستن از resident set پروسه حذف می‌شن.
→ این شبیه سیاست **LRU (Least Recently Used)** هست.
3. اجرای پروسه فقط در صورت وجود Working Set
یک پروسه اجازه اجرا پیدا می‌کنه فقط اگر تمام صفحات Working Setش در حافظه اصلی باشن.
→ این کار باعث کاهش page fault شدید (thrashing) میشه.

⚠ مشکلات این استراتژی

1. گذشته همیشه آینده رو پیش‌بینی نمی‌کنه
یعنی اینکه صفحاتی که اخیراً استفاده شدن، الزاماً دوباره استفاده نشن.
2. اندازه و ترکیب Working Set تغییر می‌کنه
پروسه ممکنه در یک لحظه کوچیک باشه (مثلاً حلقه‌ی کوچیک) و لحظه‌ی بعد خیلی بزرگ بشه (پرش به بخش جدید برنامه).
3. اندازه‌ی Δ (پنجره زمانی) مشخص نیست
انتخاب مقدار بهینه برای Δ سخت هست چون:
 - مقدار مناسب از پروسه‌ای به پروسه دیگه فرق داره.
 - حتی در طول اجرای یک پروسه هم ممکنه تغییر کنه.

نتیجه: ✓

thrashing خیلی ایده‌ی قدرتمندی برای جلوگیری از Working Set Strategy ولی پیاده‌سازی دقیق و بهینه‌ش خیلی پرهزینه و پیچیده هست چون نیاز به ردیابی دائمی صفحات و انتخاب Δ داره.

End of Chapter 10 :)